ĐẠI HỌC HOA SEN

TÔN TRỌNG SỰ KHÁC BIỆT

Data Visualization with matplotlib and Seaborn

Faculty of Information Technology

ĐẠI HỌC HOA SEN

Objectives

- ✓ The matplotlib Library
- ✓ The matplotlib Architecture
- ✓ pyplot
- ✓ Using kwargs
- ✓ Line Charts, Histograms, Bar Charts, Pie Charts, …
- ✓ The Seaborn Library

Data Visualization

- **Data Visualization** is the process of representing data in graphical or visual formats—such as charts, graphs, dashboards, and maps—to make patterns, trends, relationships, and insights easier to understand.
- In **Data Analytics**, it plays a critical role in transforming raw data into meaningful information that supports decision-making.

Data Visualization = Turning data into visuals so humans can quickly understand insights.

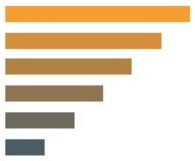
- Why Data Visualization Matters
  - **Simplifies complex data** – Large datasets become understandable.
  - **Reveals patterns & trends** – Spot growth, decline, seasonality.
  - **Supports decision-making** – Managers act faster with visual insights.
  - **Improves communication** – Easier to explain findings to non-technical audiences.
  - **Detects anomalies** – Identify outliers or unusual behavior.

Data relationships

Nominal comparison
This is a simple comparison of the quantitative values of subcategories.
Example – Number of employees.

Time series
This tracks changes in values of a consistent metric over time.
Example – Monthly sales.

Correlation
This is data with 2 or more variables that may demonstrate a positive or negative correlation to each other.
Example – Sick leave and employee wellbeing.



Ranking
This shows how 2 or more values compare to each other in relative magnitude.
Example – Annual sales.

Deviation
This examines how data points relate to each other, particularly how far any given data point differs from the mean.
Example – Budget versus actual spending.

Distribution
This shows data distribution, often around a central value.
Example – Performance results meeting targets.

Part-to-whole
This shows a subset of data compared to the larger whole.
Example – % of different categories.

- In Data Analytics, visualization helps to:
  - Identify trends (e.g., sales growth over time)
  - Compare categories (e.g., product performance)
  - Detect anomalies (e.g., sudden revenue drop)
  - Discover relationships (e.g., marketing spend vs revenue)
  - Communicate findings clearly to stakeholders
- Without visualization, analytics remains technical and difficult to interpret.

- Common Types of Data Visualizations

Chart Type	Purpose	Example Use
Bar Chart	Compare categories	Sales by product
Line Chart	Show trends over time	Monthly revenue
Pie Chart	Show proportions	Market share
Scatter Plot	Show relationships	Price vs demand
Histogram	Show distribution	Customer age distribution
Heatmap	Show intensity patterns	Correlation matrix
Dashboard	Combine multiple visuals	Executive reporting

Chart	Visual	X axis	Y axis	Analysis	Example
Scatter plot/Line Plot		Continuous	Continuous	- Understanding linear, non-linear relationship between two variables - Trend analysis, change in KPI over time	- How does heart rate change with age? - How sales of a company varied over a period of time?
Bar Graph		Categorical	Continuous	- How Y (can be any performance indicator) varies across different categories?	- How sales in 2019 varied for different mobile phone brands? i.e. mobile phone brand is the category and sales is the KPI
Stack Bar Graph		Categorical	Continuous	- Relative comparison of multiple categories within a category	- Comparison of revenue generated by Apple, Samsung & Xiaomi across different products like mobile phone, laptops, television, and headsets
Box Plot		Continuous		- Outlier detection - Analysing data distribution across Median and Inter Quartile Range	- How different sales figures across a year is distributed?
Pie Chart		Categorical & Continuous		- Relative comparison of different categories for one single entity in terms of proportion/percentages	- What percentage of Sales in 2019 is constituted by different products under Apple?
Histogram Plot		Continuous	-	- How distribution of values of x varies across different range buckets?	- Distribution of income across income buckets for developing countries

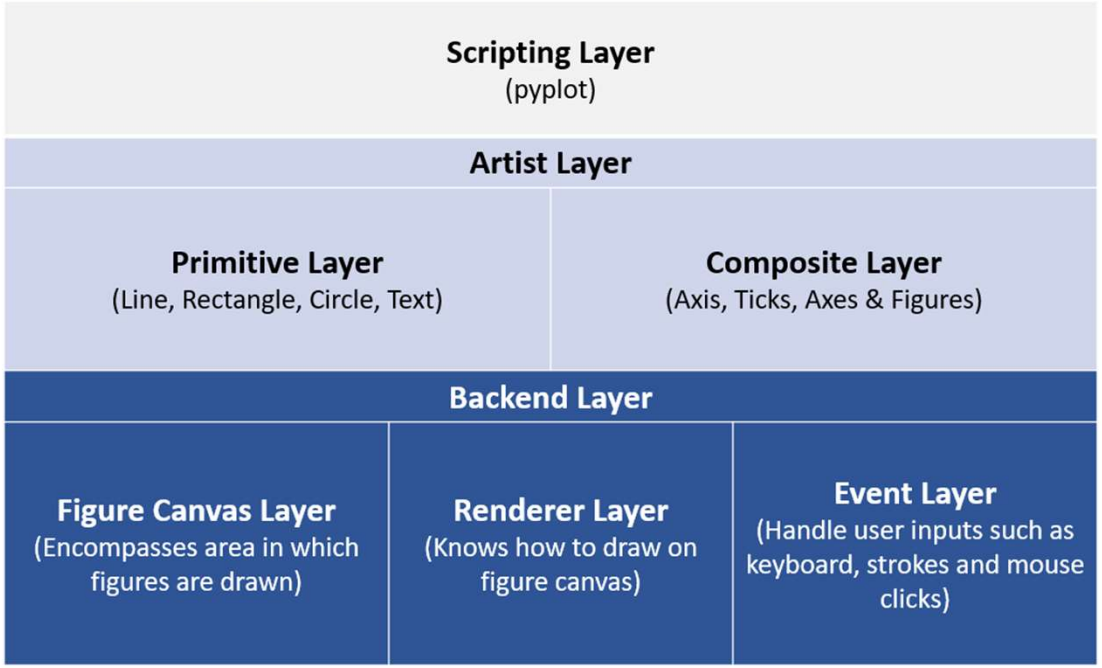
- **Popular Data Visualization Tools**
  - Microsoft Power BI
  - Tableau
  - Qlik Sense
  - Microsoft Excel
  - **Python** (Matplotlib, Seaborn, Plotly)
  - R (ggplot2)

What is Matplotlib?

- **Matplotlib** is:
  - An open-source Python library
  - Used for creating charts and graphs
  - Widely applied in research, AI, business analytics
  - The foundation of many Python visualization tools

The matplotlib Architecture

- The **Matplotlib architecture** describes how the library is structured internally to transform Python code into a visual chart.
- It is typically explained as a **3-layer system**:
  - Scripting
  - Artist
  - Backend



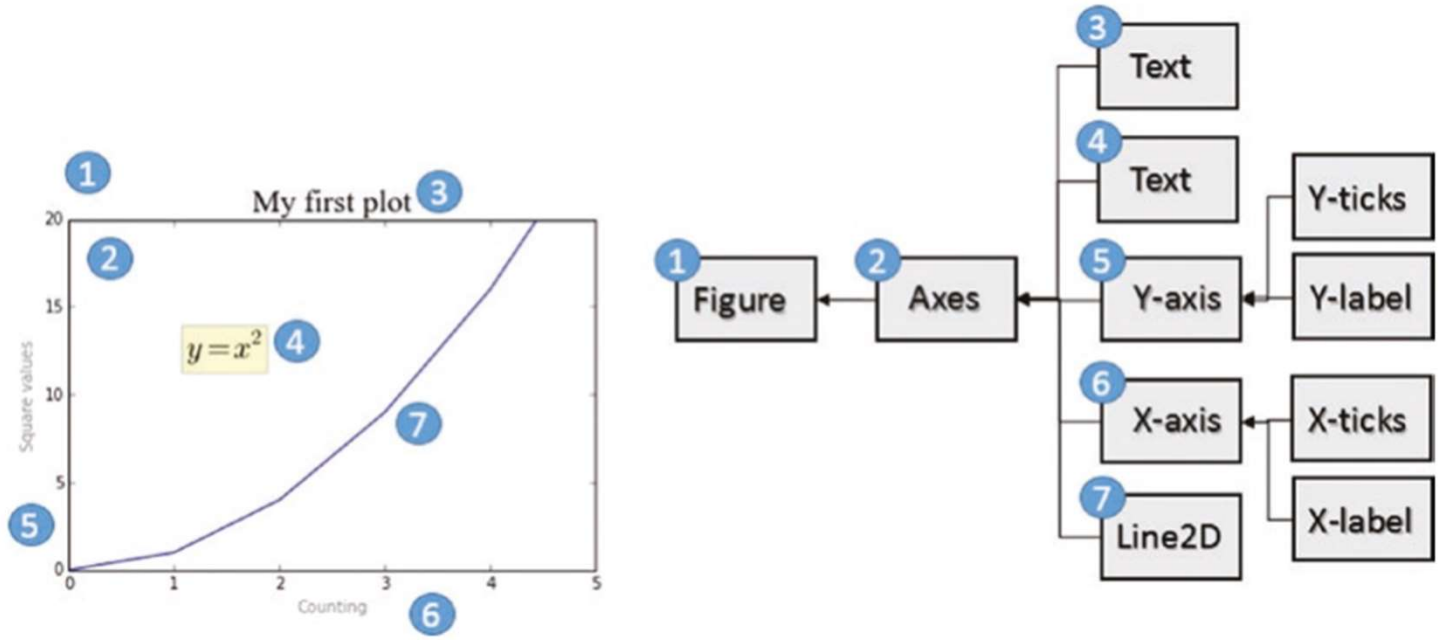
Backend Layer

- The Backend layer is the lowest level in matplotlib's architecture and contains three main components:
- **FigureCanvas**: Represents the drawing area/surface where graphics are rendered
- **Renderer**: Handles the actual drawing operations on the FigureCanvas
- **Event**: Manages user interactions including keyboard and mouse inputs
- This layer provides the fundamental low-level APIs and classes that implement basic graphic elements and handle the core drawing functionality.

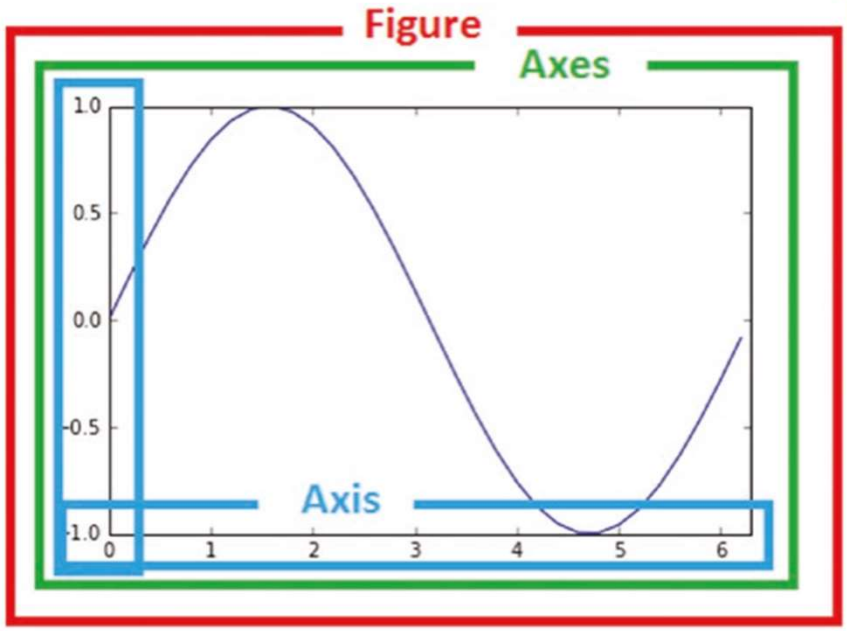
Artist Layer

- The Artist layer is the intermediate layer where all chart elements (title, axis labels, markers, etc.) exist as Artist objects in a hierarchical structure. It consists of two types:
  - **Primitive Artists**: Basic individual elements that form graphical representations
    - Examples: Line2D, Rectangle, Circle, text elements
  - **Composite Artists**: Complex elements built from multiple primitive artists
    - Examples: Axis, Ticks, Axes, and Figures
- Each Artist instance plays a specific role within the hierarchy, with composite artists containing and organizing primitive artists to create complete chart visualizations.

- Each element of a chart corresponds to an instance of an Artist structured in a hierarchy



- The three main artist objects in the hierarchy of the Artist layer
  - **Figure**: The top-level container in the hierarchy
    - Represents the entire graphical representation
    - Can contain multiple Axes objects
  - **Axes**: The actual plot or chart area
    - Belongs to a single Figure
    - Contains two Axis objects (three for 3D plots)
    - Includes other elements like title, x-label, and y-label
  - **Axis**: Handles numerical representation on Axes
    - Manages axis limits
    - Controls ticks (axis marks) through **Locator** objects
    - Formats tick labels through **Formatter** objects



Scripting Layer (pyplot): A high-level interface designed for data analysis and visualization

- Provides a simpler, more intuitive interface compared to the lower-level Artist classes and matplotlib API
- Best suited for:
  - Data analysis and visualization tasks
  - Scientific computing workflows
- While the Artist layer and matplotlib API are better for:
  - Web application servers
  - GUI development
- The pyplot interface makes common plotting operations more accessible and streamlined for users focused on data exploration and visualization rather than application development.

- **pyplot**: An internal module within matplotlib that provides a MATLAB-like interface for plotting
- **pylab**: A separate module installed alongside matplotlib that combines pyplot functionality with NumPy features

- Key differences:
  - **pyplot** is more focused and is the recommended modern approach
  - **pylab** merges matplotlib (pyplot) and NumPy into a single namespace, mimicking MATLAB's environment
- While both are referenced in documentation, pyplot is generally preferred for clearer code and explicit namespace management

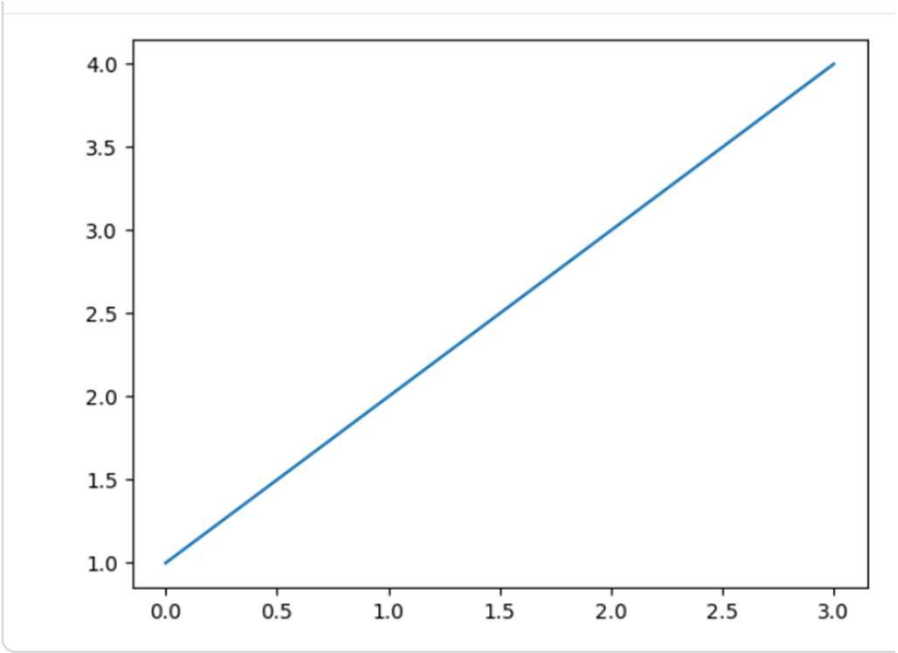
```
from pylab import *
import matplotlib.pyplot as plt
import numpy as np
```

pyplot

```
import matplotlib.pyplot as plt
```

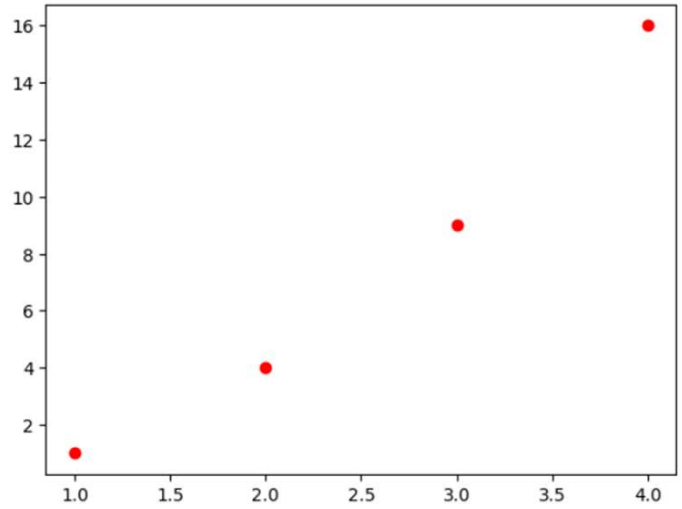
- pyplot is a **high-level plotting interface** in Matplotlib that provides a simple, MATLAB-like way to create visualizations in Python.

```
plt.plot([1,2,3,4])
plt.show()
```



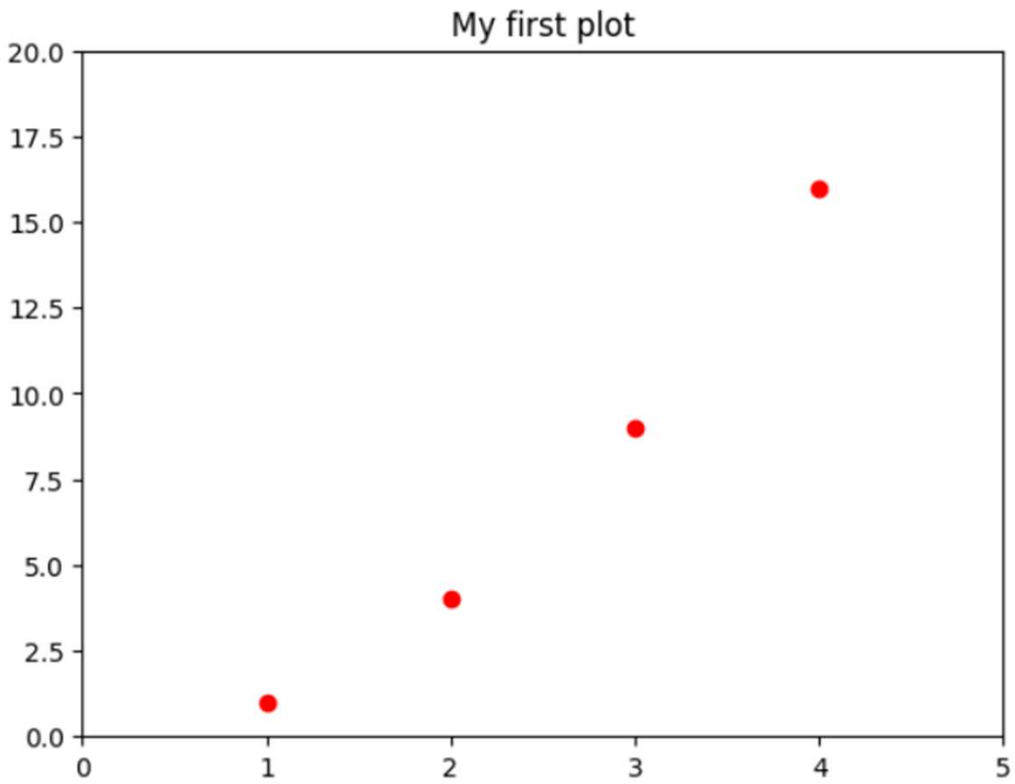
- Set the Properties of the Plot
  - the plot() function's third argument and default behavior:
  - **Default plot() behavior:**
    - Axes size matches input data range
    - No title or axis labels
    - No legend
    - Blue line connecting points
  - **Third argument functionality:**
    - Accepts formatting strings to customize point representation
    - Overrides default plotting style
    - Example: 'ro' creates red dots for each (x,y) pair instead of connected blue line

```
plt.plot([1,2,3,4],[1,4,9,16],'ro')
plt.show()
```



- Another possible change to the default behavior would be to define a range both on the x-axis and on the y-axis within a list [xmin, xmax, ymin, ymax] and then pass it as an argument to the **axis()** function.

```
plt.axis([0,5,0,20])
plt.title('My first plot')
plt.plot([1,2,3,4],[1,4,9,16],'ro')
plt.show()
```

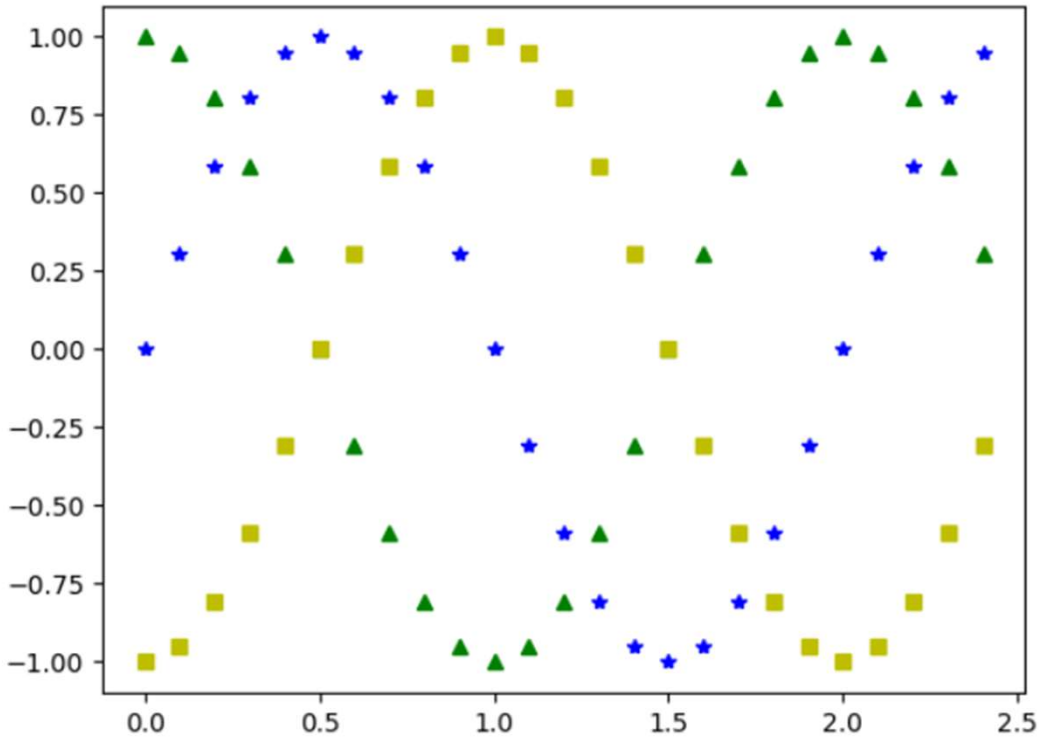


matplotlib and NumPy

- **NumPy Foundation:**
  - matplotlib is built on NumPy, even as a graphical library
  - Input lists are internally converted to NumPy arrays
  - NumPy arrays can be passed directly as input data without conversion
- **Practical Example - Plotting Sin Function:**
  - Import sin() from math module
  - Generate x-axis points using NumPy's arange()
  - Apply sin() to all array items using map() (avoiding for loops)
  - Plot multiple trends on the same figure

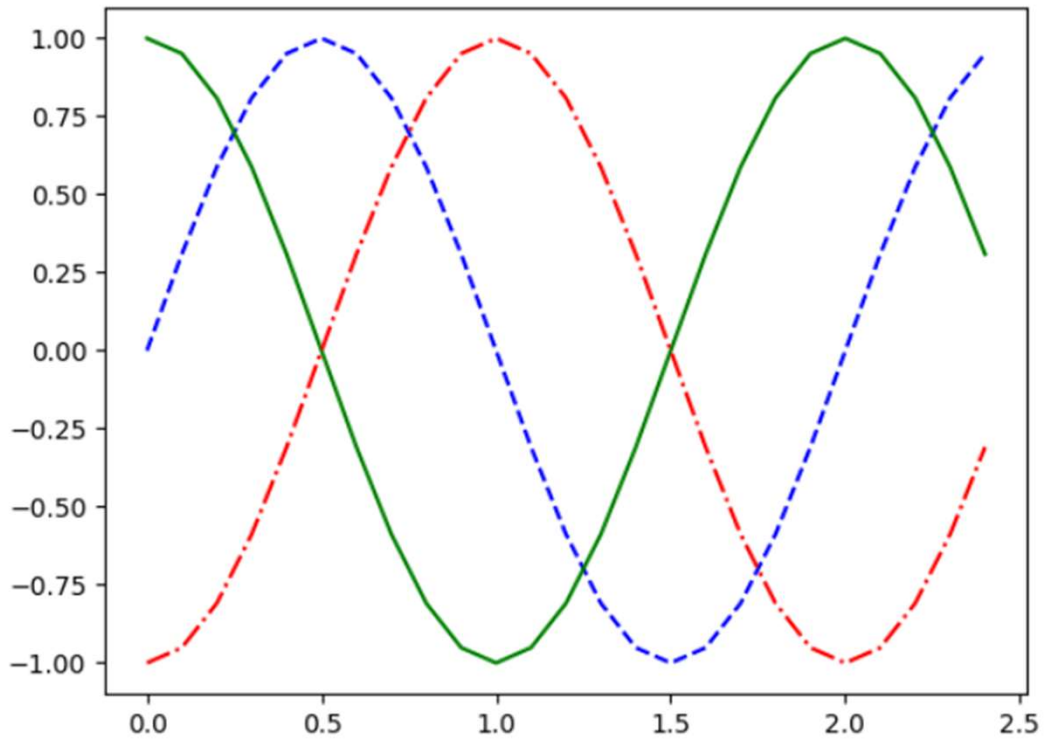
```
import math
import numpy as np
```

```
t = np.arange(0,2.5,0.1)
y1 = np.sin(math.pi*t)
y2 = np.sin(math.pi*t+math.pi/2)
y3 = np.sin(math.pi*t-math.pi/2)
plt.plot(t,y1,'b*',t,y2,'g^',t,y3,'ys')
plt.show()
```



Three sinusoidal trends phase-shifted by π / 4 represented by markers

```
plt.plot(t,y1,'b--',t,y2,'g',t,y3,'r-.')
plt.show()
```

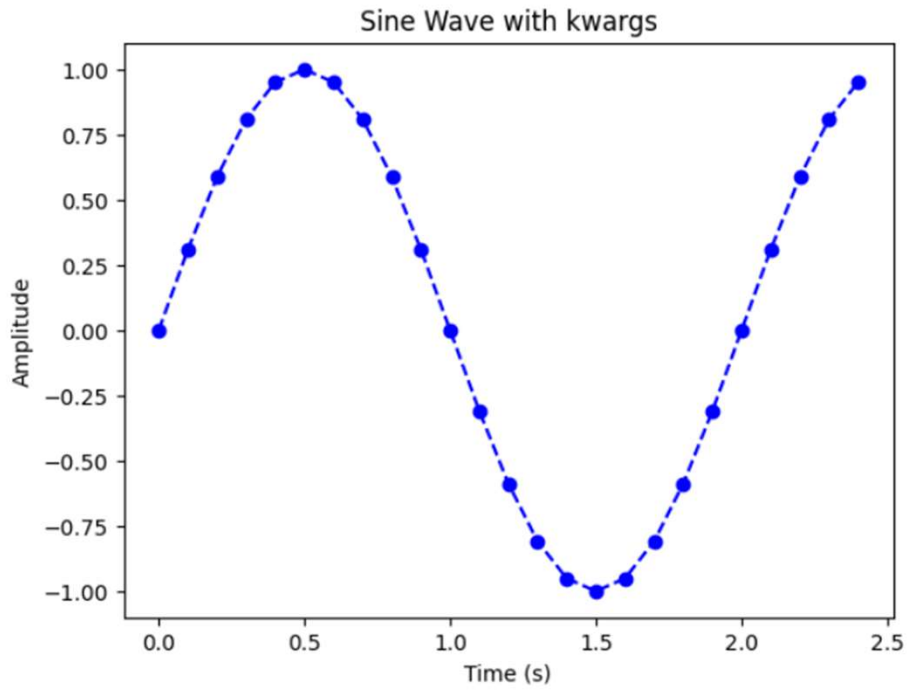


This chart represents the three sinusoidal patterns with colored lines

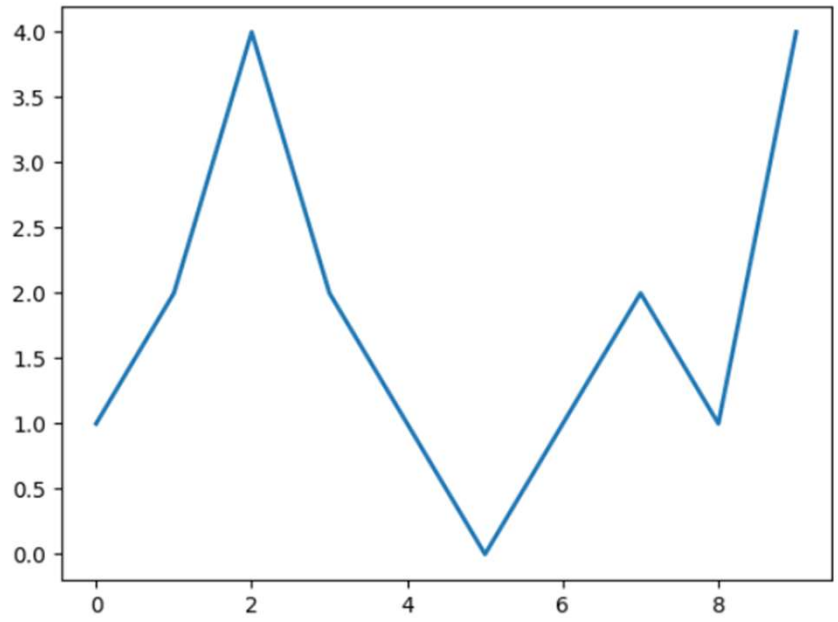
Using kwargs

- **kwargs** is a special syntax in Python that allows functions to accept an arbitrary number of keyword arguments.
  - **kwargs** stands for "keyword arguments"
  - The double asterisk (**) collects extra keyword arguments into a dictionary
  - The name "kwargs" is conventional but not required (though widely used)
- **In matplotlib:**
  - Many plotting functions accept **kwargs to customize appearance
  - Examples: color, linewidth, marker size, transparency, etc.
- **Common matplotlib kwargs:**
  - color or c: Line or marker color
  - linewidth or lw: Line thickness
  - linestyle or ls: Line pattern (solid, dashed, etc.)
  - marker: Point marker style
  - alpha: Transparency (0-1)
  - label: Legend label

```
plt.plot(t, y1, color='blue', linestyle='--', marker='o')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Sine Wave with kwargs')
plt.show()
```




```
plt.plot([1,2,4,2,1,0,1,2,1,4],linewidth=2.0)
plt.show()
```

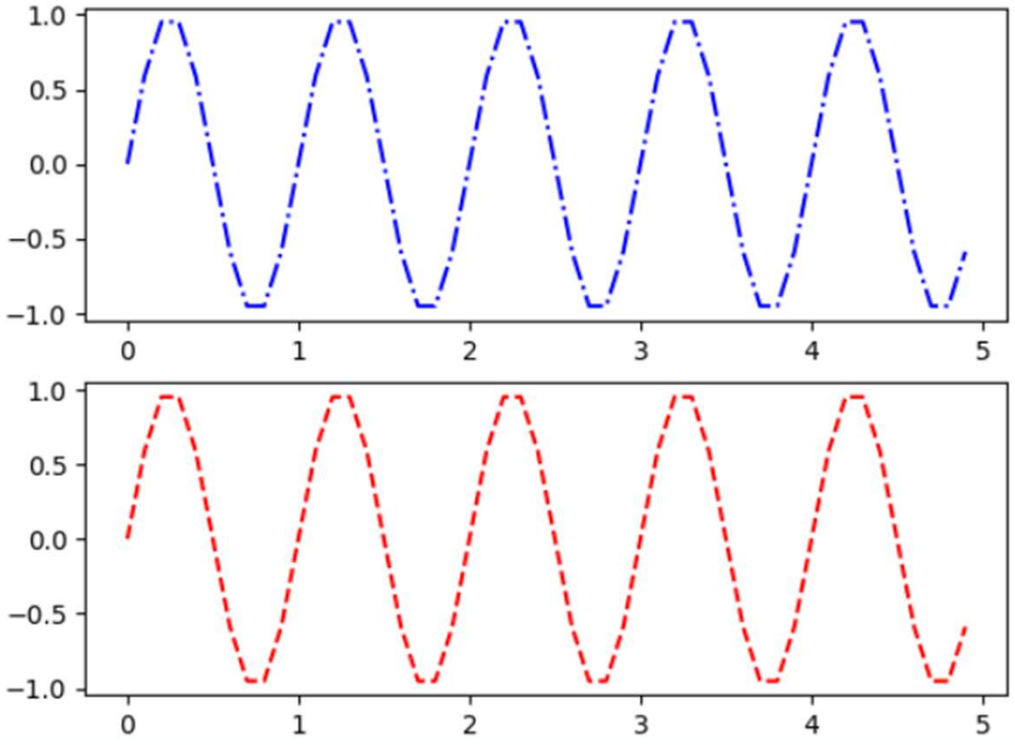


Working with Multiple Figures and Axes

- **Multiple Figures and Subplots:**
  - matplotlib can manage multiple figures simultaneously
  - Each figure can contain multiple subplots (different plots within the same figure)
- **Current Figure and Current Axes:**
  - pyplot maintains concepts of "current" Figure and "current" Axes
  - Commands are routed to the currently active Figure and subplot
- **subplot() Function:**
  - Divides a figure into multiple drawing areas
  - Sets the current active subplot for subsequent commands
  - Takes three integer arguments:
    - **First number**: Vertical divisions (rows)
    - **Second number**: Horizontal divisions (columns)
    - **Third number**: Selects which subplot becomes current (1-indexed, reading left to right, top to bottom)

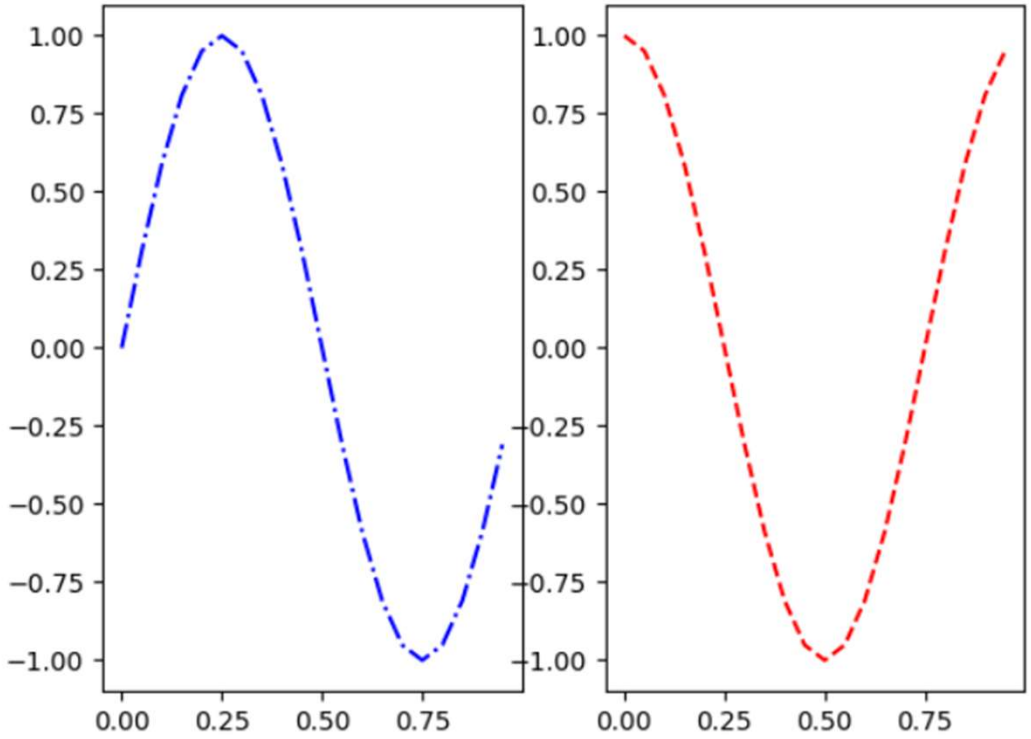
- Display two sinusoidal trends (sine and cosine) and the best way to do that is to divide the canvas vertically in two horizontal subplots

```
t = np.arange(0,5,0.1)
y1 = np.sin(2*np.pi*t)
y2 = np.sin(2*np.pi*t)
plt.subplot(211)
plt.plot(t,y1,'b-.')
plt.subplot(212)
plt.plot(t,y2,'r--')
plt.show()
```



- Do the same thing by dividing the figure into two vertical subplots.

```
t = np.arange(0.,1.,0.05)
y1 = np.sin(2*np.pi*t)
y2 = np.cos(2*np.pi*t)
plt.subplot(121)
plt.plot(t,y1,'b-.')
plt.subplot(122)
plt.plot(t,y2,'r--')
plt.show()
```

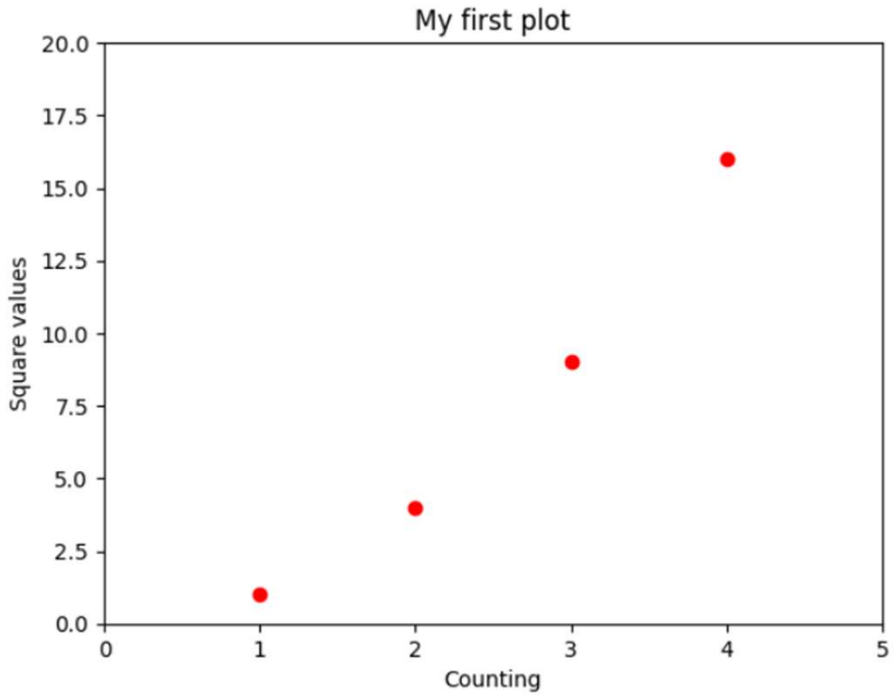


Adding Elements to the Chart

To add elements, such as text labels, a legend, and so on, to a chart.

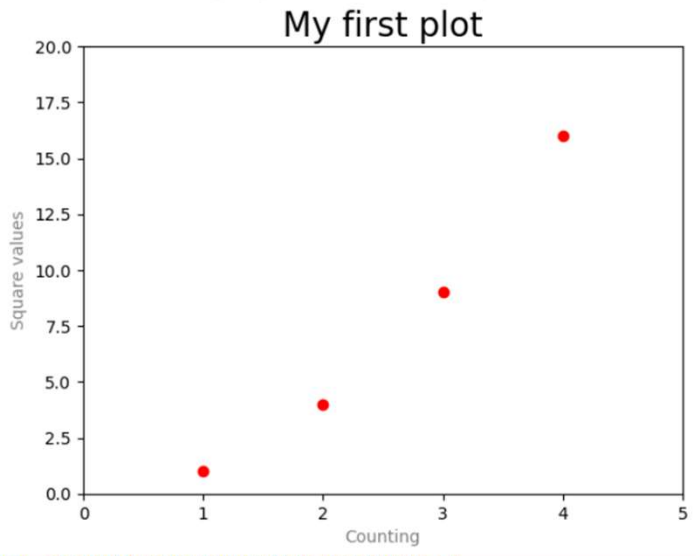
Adding Text

```
plt.axis([0,5,0,20])
plt.title('My first plot')
plt.xlabel('Counting')
plt.ylabel('Square values')
plt.plot([1,2,3,4],[1,4,9,16],'ro')
plt.show()
```

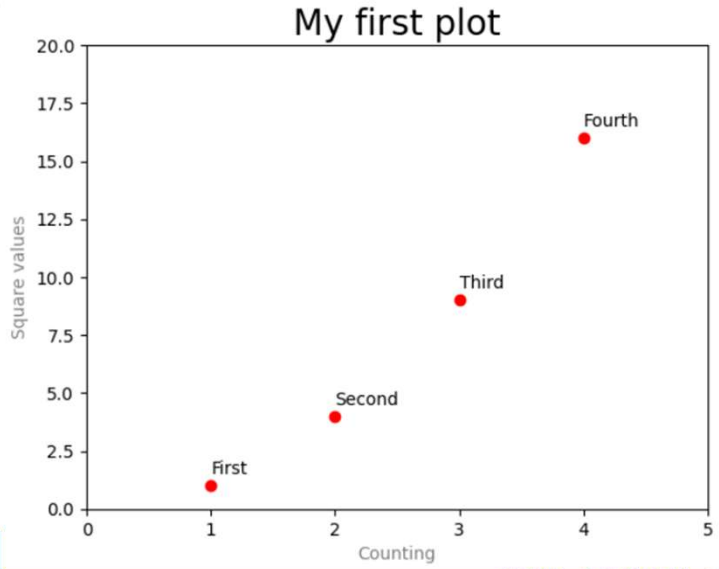


```
plt.axis([0,5,0,20])
plt.title('My first plot', fontsize=20, fontname='Arial')
plt.xlabel('Counting', color='gray')
plt.ylabel('Square values',color='gray')
plt.plot([1,2,3,4],[1,4,9,16],'ro')
plt.show()
```

```
WARNING:matplotlib.font_manager:findfont: Font family 'Arial' not found.
WARNING:matplotlib.font_manager:findfont: Font family 'Arial' not found.
WARNING:matplotlib.font_manager:findfont: Font family 'Arial' not found.
WARNING:matplotlib.font_manager:findfont: Font family 'Arial' not found.
WARNING:matplotlib.font_manager:findfont: Font family 'Arial' not found.
```



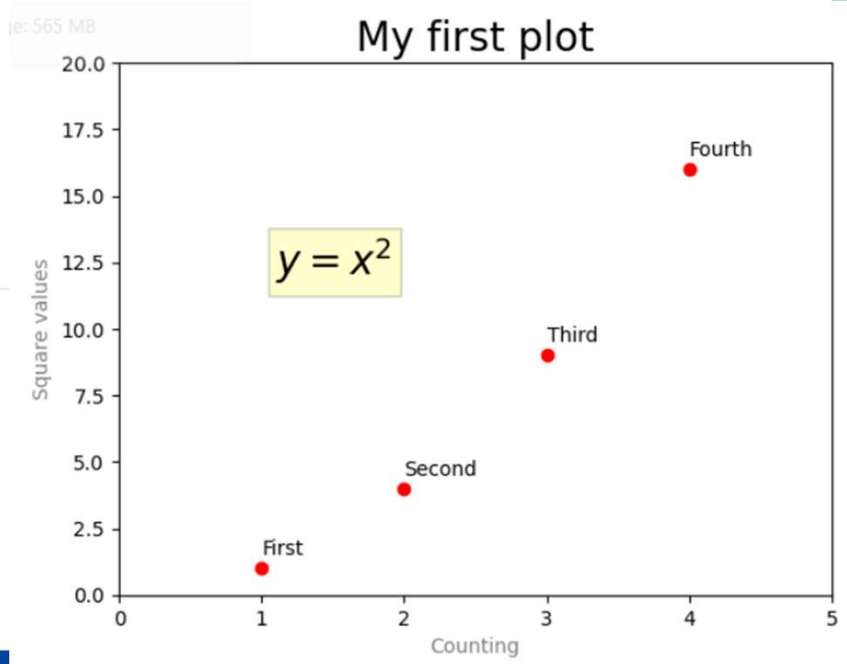

```
plt.axis([0,5,0,20])
plt.title('My first plot', fontsize=20, fontname='Times New Roman')
plt.xlabel('Counting', color='gray')
plt.ylabel('Square values',color='gray')
plt.text(1,1.5,'First')
plt.text(2,4.5,'Second')
plt.text(3,9.5,'Third')
plt.text(4,16.5,'Fourth')
plt.plot([1,2,3,4],[1,4,9,16],'ro')
plt.show()
```



Every point of the plot has an informative label

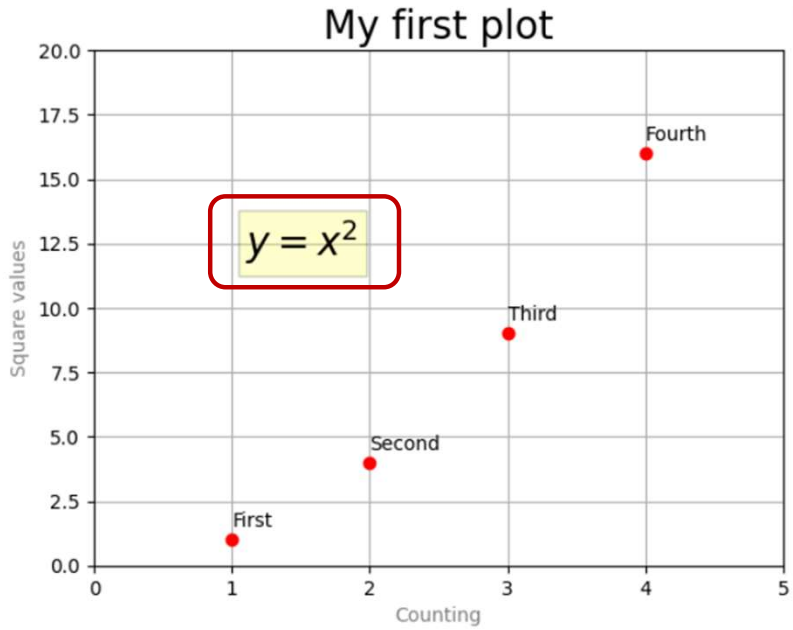
- can add the formula describing the trend followed by the point of the plot and enclose it in a colored bounding box

```
plt.axis([0,5,0,20])
plt.title('My first plot', fontsize=20)
plt.xlabel('Counting', color='gray')
plt.ylabel('Square values',color='gray')
plt.text(1,1.5,'First')
plt.text(2,4.5,'Second')
plt.text(3,9.5,'Third')
plt.text(4,16.5,'Fourth')
plt.text(1.1,12,r'$y = x^2$', fontsize=20, bbox={'facecolor':'yellow','alpha':0.2})
plt.plot([1,2,3,4],[1,4,9,16],'ro')
plt.show()
```



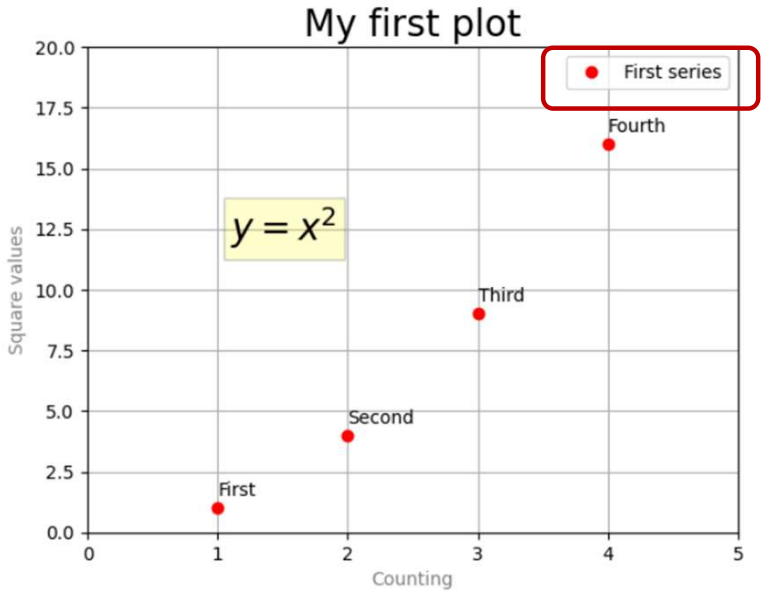
Adding a Grid

```
plt.axis([0,5,0,20])
plt.title('My first plot', fontsize=20)
plt.xlabel('Counting', color='gray')
plt.ylabel('Square values',color='gray')
plt.text(1,1.5,'First')
plt.text(2,4.5,'Second')
plt.text(3,9.5,'Third')
plt.text(4,16.5,'Fourth')
plt.text(1.1,12,r'$y = x^2$', fontsize=20, bbox={'facecolor':'yellow','alpha':0.2})
plt.plot([1,2,3,4],[1,4,9,16],'ro')
plt.grid(True)
plt.show()
```



Adding a Legen

```
plt.axis([0,5,0,20])
plt.title('My first plot', fontsize=20)
plt.xlabel('Counting', color='gray')
plt.ylabel('Square values',color='gray')
plt.text(1,1.5,'First')
plt.text(2,4.5,'Second')
plt.text(3,9.5,'Third')
plt.text(4,16.5,'Fourth')
plt.text(1.1,12,r'$y = x^2$', fontsize=20, bbox={'facecolor':'yellow','alpha':0.2})
plt.plot([1,2,3,4],[1,4,9,16],'ro')
plt.grid(True)
plt.legend(['First series'])
plt.show()
```

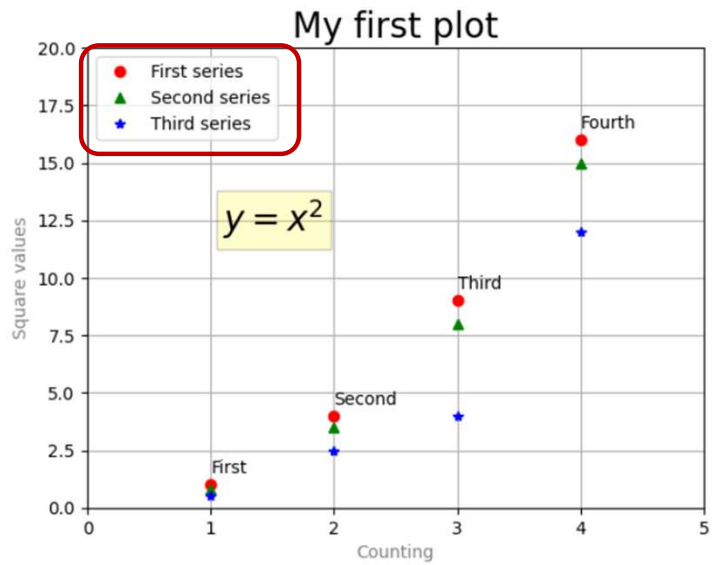


- Legend positioning in matplotlib
  - **Default Legend Behavior:**
    - Legend is added to the **upper-right corner** by default (loc=1)
  - **Customizing Legend Position with loc kwarg:**
    - The loc parameter accepts numbers 0-10 to position the legend
    - Each number corresponds to a specific location on the chart
  - **Common loc values:**
    - **loc=1**: Upper-right corner (default)
    - **loc=2**: Upper-left corner
    - **loc=3**: Lower-left corner
    - **loc=4**: Lower-right corner
    - **loc=0**: "Best" position (automatically chooses location to minimize overlap)

Location Code	Location String
0	best
1	upper-right
2	upper-left
3	lower-right
4	lower-left
5	right
6	center-left
7	center-right
8	lower-center
9	upper-center
10	center

The Possible Values for the loc Keyword

```
plt.axis([0,5,0,20])
plt.title('My first plot', fontsize=20)
plt.xlabel('Counting', color='gray')
plt.ylabel('Square values',color='gray')
plt.text(1,1.5,'First')
plt.text(2,4.5,'Second')
plt.text(3,9.5,'Third')
plt.text(4,16.5,'Fourth')
plt.text(1.1,12,r'$y = x^2$', fontsize=20, bbox={'facecolor':'yellow','alpha':0.2})
plt.grid(True)
plt.plot([1,2,3,4],[1,4,9,16],'ro')
plt.plot([1,2,3,4],[0.8,3.5,8,15],'g^')
plt.plot([1,2,3,4],[0.5,2.5,4,12],'b*')
plt.legend(['First series','Second series','Third series'], loc=2)
plt.show()
```

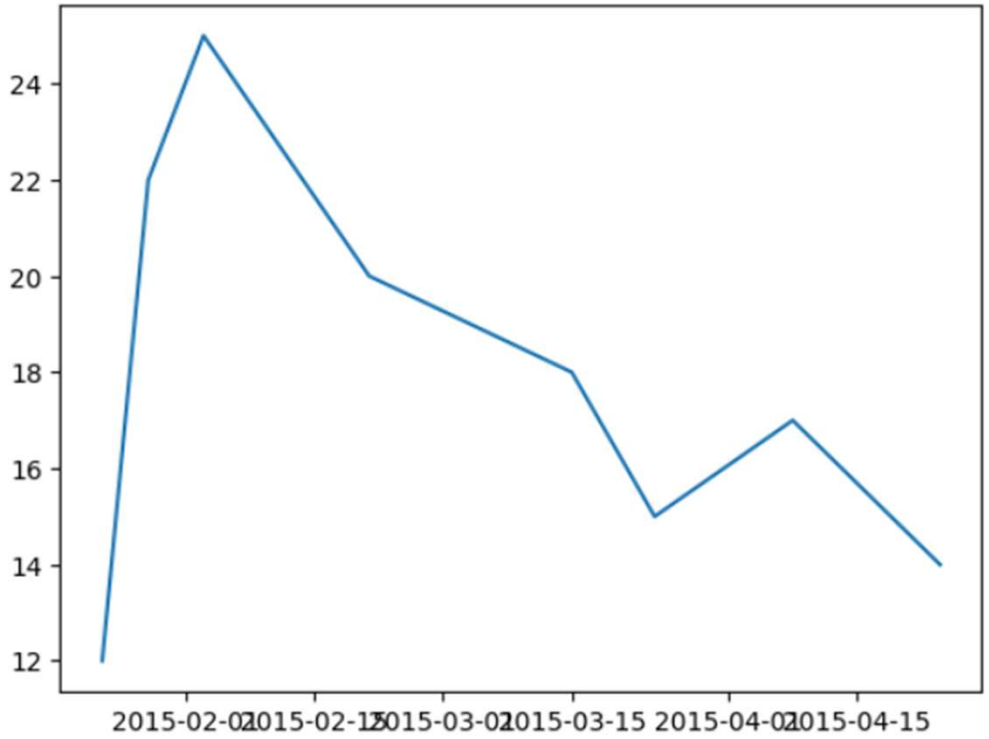


Handling Date Values

- **The Challenge:**
  - Date-time data is common in data analysis
  - Displaying dates along axes (especially x-axis) can be problematic
  - Main difficulty lies in managing tick marks and labels appropriately
- **Common Issues:**
  - **Overlapping labels** when too many dates are shown
  - **Poor spacing** that doesn't reflect time intervals
  - **Illegible formatting** (e.g., full timestamps instead of simplified dates)
  - **Automatic tick selection** that may not suit your data frequency

```
import datetime
import numpy as np
import matplotlib.pyplot as plt
```

```
events = [datetime.date(2015,1,23),
          datetime.date(2015,1,28),
          datetime.date(2015,2,3),
          datetime.date(2015,2,21),
          datetime.date(2015,3,15),
          datetime.date(2015,3,24),
          datetime.date(2015,4,8),
          datetime.date(2015,4,24)]
readings = [12,22,25,20,18,15,17,14]
plt.plot(events,readings)
plt.show()
```

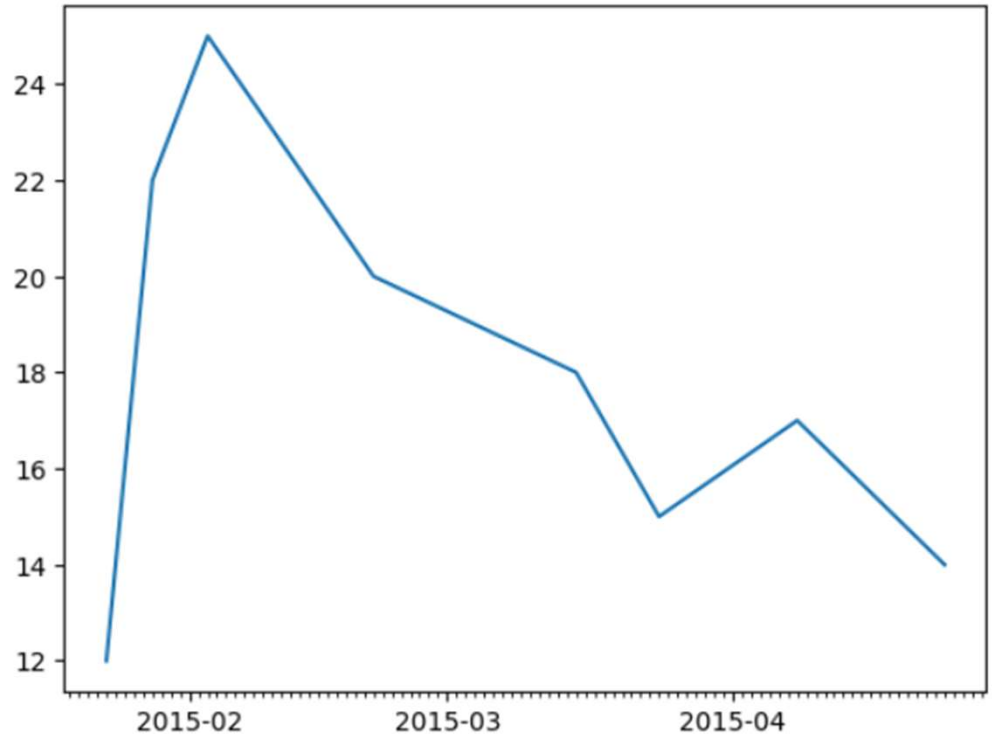


If they aren't handled, displaying date-time values can be problematic

- **Key Steps**
- **Import the dates module**
  - Use matplotlib.dates to handle time-based data properly.
- **Define time scales**
  - Use:
    - MonthLocator() → for monthly ticks
    - DayLocator() → for daily ticks
  - These define how dates are spaced along the axis.
- **Format tick labels**
  - Use DateFormatter() to control how dates appear.
  - Keep labels simple (e.g., year-month) to avoid clutter and overlapping text.
- **Apply locators and formatter**
  - Set tick positions:
    - set_major_locator() → typically for months
    - set_minor_locator() → typically for days
  - Set label formatting:
    - set_major_formatter() → formats the major tick labels (e.g., months)

```
import matplotlib.dates as mdates
```

```
months = mdates.MonthLocator()
days = mdates.DayLocator()
timeFmt = mdates.DateFormatter('%Y-%m')
events = [datetime.date(2015,1,23),
          datetime.date(2015,1,28),
          datetime.date(2015,2,3),
          datetime.date(2015,2,21),
          datetime.date(2015,3,15),
          datetime.date(2015,3,24),
          datetime.date(2015,4,8),
          datetime.date(2015,4,24)]
readings = [12,22,25,20,18,15,17,14]
fig, ax = plt.subplots()
plt.plot(events, readings)
ax.xaxis.set_major_locator(months)
ax.xaxis.set_major_formatter(timeFmt)
ax.xaxis.set_minor_locator(days)
plt.show()
```



Line Chart

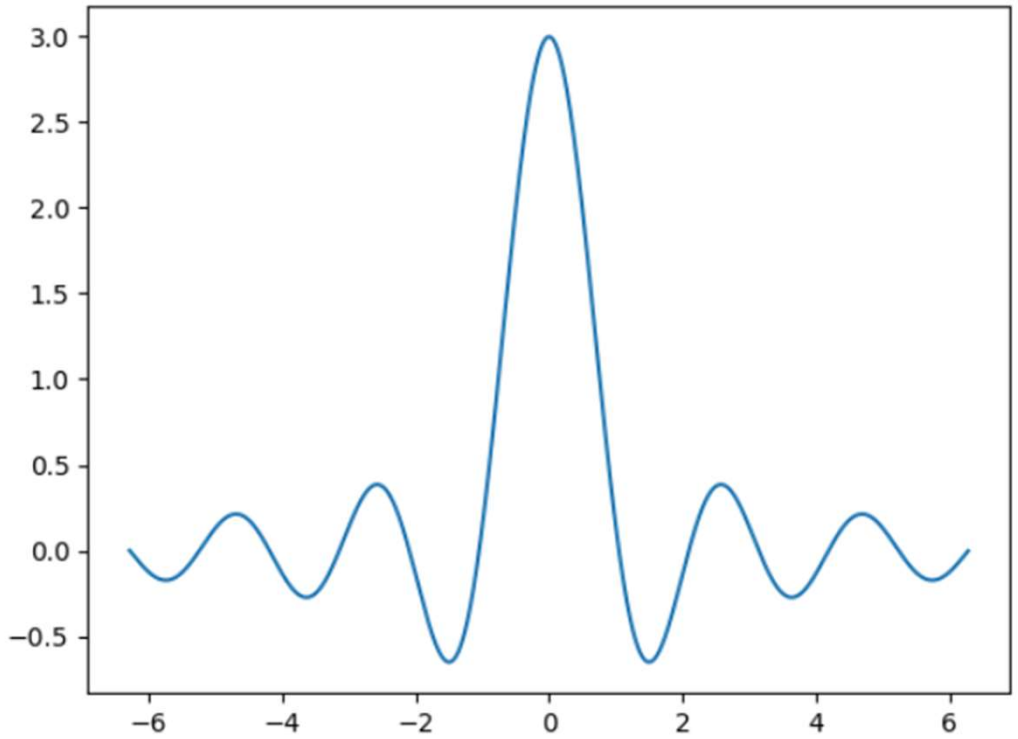
- A graphical representation where data points are connected by straight line segments to visualize the relationship between two variables, often showing trends over a continuous interval like time
- **Key Characteristics**
  - **Trend Visualization**: Line charts are commonly used to display how data changes over time, helping to identify trends and patterns.
  - **Axes**: Typically, the independent variable (like time) is plotted on the horizontal (x) axis, and the dependent variable (like temperature or stock price) is plotted on the vertical (y) axis.
  - **Data Points**: Data points, or "markers," can be displayed on the lines to highlight exact values.
  - **Multiple Series**: Multiple lines can be plotted on the same chart to compare different trends simultaneously

With matplotlib

```
import matplotlib.pyplot as plt
```

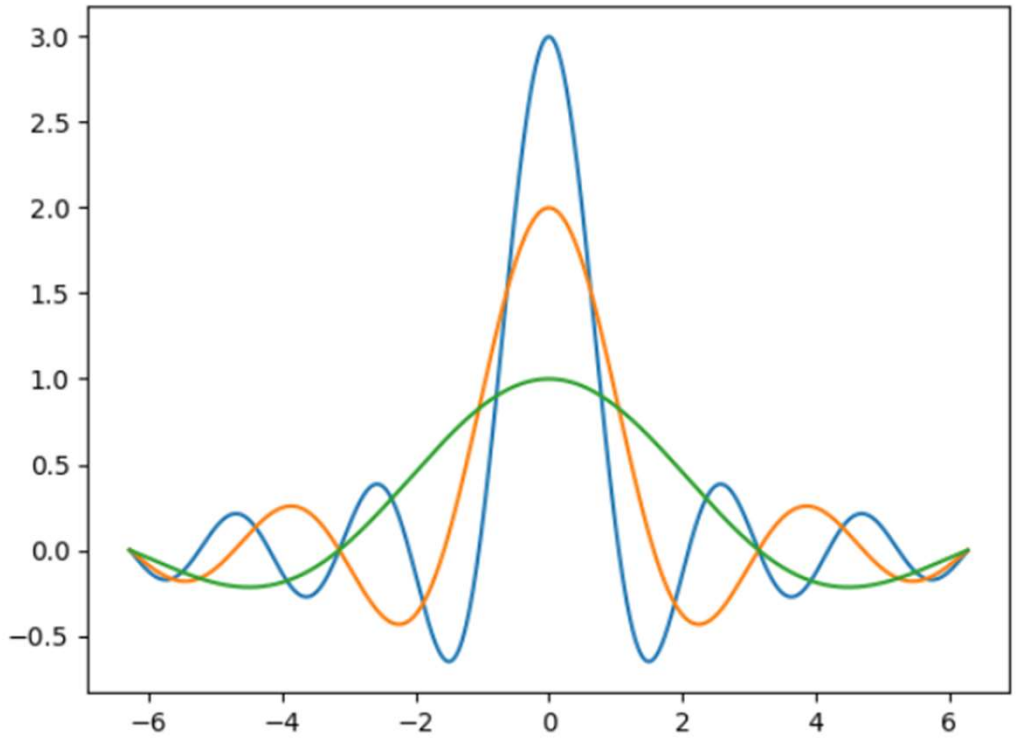
$$y = sin(3^{*}x)/x$$

```
x = np.arange(-2*np.pi,2*np.pi,0.01)
y = np.sin(3*x)/x
plt.plot(x,y)
plt.show()
```



$$y = sin(n^*x)/x$$

```
x = np.arange(-2*np.pi,2*np.pi,0.01)
y = np.sin(3*x)/x
y2 = np.sin(2*x)/x
y3 = np.sin(x)/x
plt.plot(x,y)
plt.plot(x,y2)
plt.plot(x,y3)
plt.show()
```



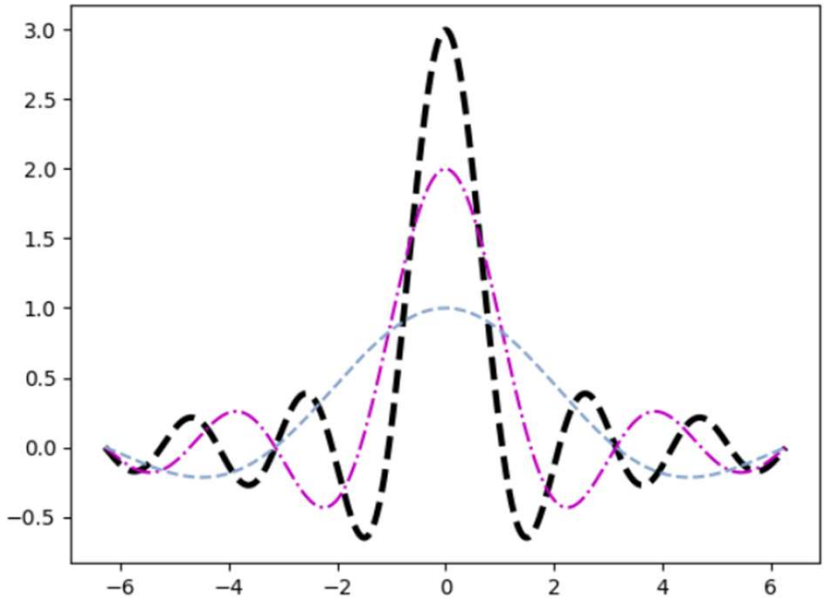
Three different series are drawn with different colors in the same chart

- As the third argument of the plot() function, you can specify some codes that correspond to the color and other codes that correspond to line styles

Code	Color
b	blue
g	green
r	red
c	cyan
m	magenta
y	yellow
k	black
w	white

Color Codes

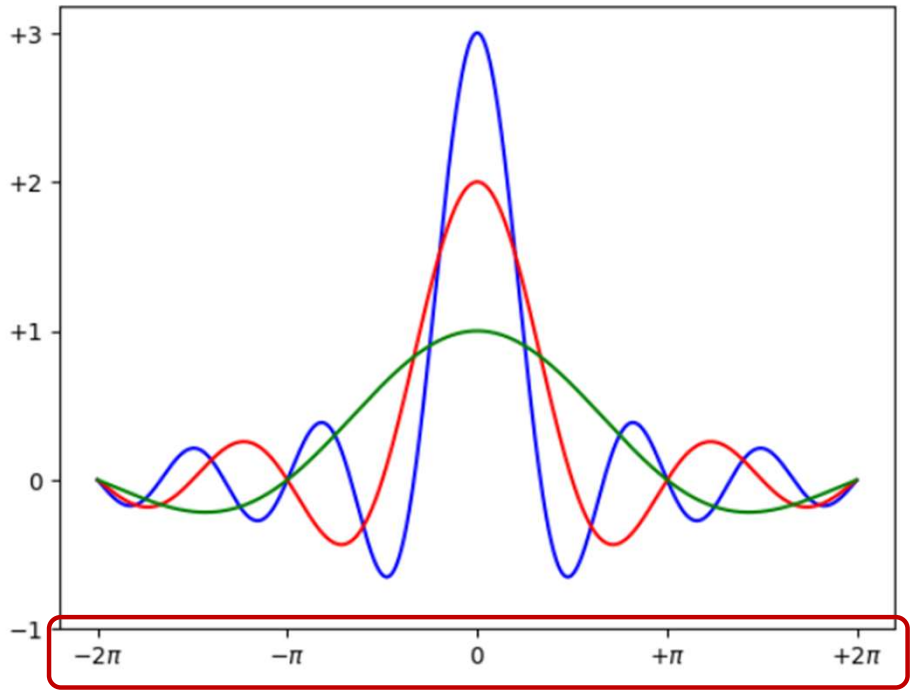
```
x = np.arange(-2*np.pi,2*np.pi,0.01)
y = np.sin(3*x)/x
y2 = np.sin(2*x)/x
y3 = np.sin(x)/x
plt.plot(x,y,'k--',linewidth=3)
plt.plot(x,y2,'m-.')
plt.plot(x,y3,color='#87a3cc',linestyle='--')
plt.show()
```



define colors and line styles using character codes

- Use the xticks() and yticks() functions: defined a range from -2π to 2π on the x-axis

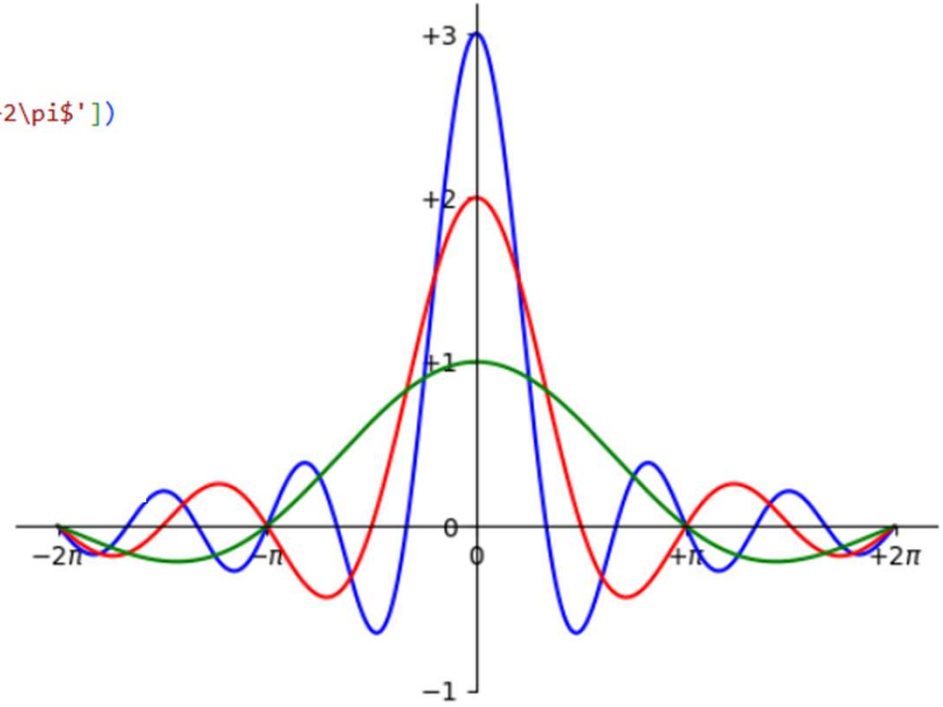
```
x = np.arange(-2*np.pi,2*np.pi,0.01)
y = np.sin(3*x)/x
y2 = np.sin(2*x)/x
y3 = np.sin(x)/x
plt.plot(x,y,color='b')
plt.plot(x,y2,color='r')
plt.plot(x,y3,color='g')
plt.xticks([-2*np.pi,-np.pi,0, np.pi, 2*np.pi],
           [r'$-2\pi$',r'$-\pi$',r'$0$',r'$+\pi$',r'$+2\pi$'])
plt.yticks([-1,0,1,2,3],
           [r'$-1$',r'$0$',r'$+1$',r'$+2$',r'$+3$'])
plt.show()
```



The tick label can be improved by adding text with LaTeX format

- An alternative layout is to make both axes intersect at the origin (0, 0), forming standard Cartesian axes in the center of the plot.
- To do this in Matplotlib:
  - Retrieve the current **Axes** object using gca().
  - Access the four sides (spines) of the bounding box: right, left, top, and bottom.
  - Hide the unnecessary sides (e.g., right and top) by setting their color to none using set_color().
  - Move the remaining spines (left and bottom) so they cross at the origin using set_position().

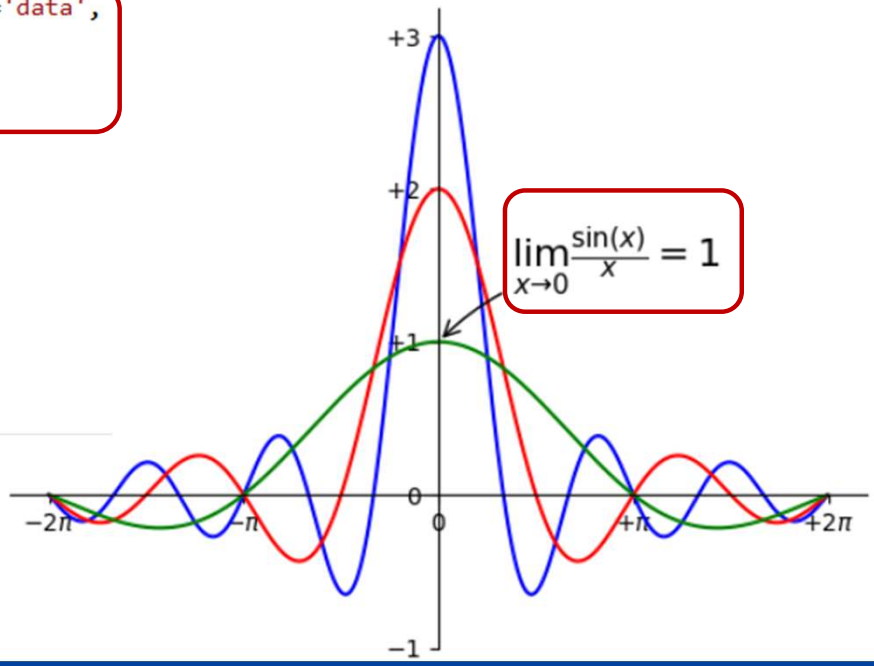
```
x = np.arange(-2*np.pi,2*np.pi,0.01)
y = np.sin(3*x)/x
y2 = np.sin(2*x)/x
y3 = np.sin(x)/x
plt.plot(x,y,color='b')
plt.plot(x,y2,color='r')
plt.plot(x,y3,color='g')
plt.xticks([-2*np.pi,-np.pi,0, np.pi, 2*np.pi],
           [r'$-2\pi$',r'$-\pi$',r'$0$',r'$+\pi$',r'$+2\pi$'])
plt.yticks([-1,0,1,2,3],
           [r'$-1$',r'$0$',r'$+1$',r'$+2$',r'$+3$'])
ax = plt.gca()
ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
ax.xaxis.set_ticks_position('bottom')
ax.spines['bottom'].set_position(('data',0))
ax.yaxis.set_ticks_position('left')
ax.spines['left'].set_position(('data',0))
plt.show()
```



The chart shows two Cartesian axes

- Matplotlib provides the annotate() function to add annotations to a plot, which is especially useful for highlighting specific points.
- Although it requires several keyword arguments (kwargs) and can be somewhat complex to configure, it allows precise customization.
- **Key Elements of annotate():**
  - **First argument**: A string (often written in LaTeX format) representing the text to display.
  - **xy kwarg**: Specifies the coordinates [x, y] of the point being highlighted.
  - **xytext kwarg**: Defines the position of the annotation text relative to the point.
  - **arrowprops kwarg**: Controls the appearance of the arrow connecting the text to the highlighted point.

```
x = np.arange(-2*np.pi,2*np.pi,0.01)
y = np.sin(3*x)/x
y2 = np.sin(2*x)/x
y3 = np.sin(x)/x
plt.plot(x,y,color='b')
plt.plot(x,y2,color='r')
plt.plot(x,y3,color='g')
plt.xticks([-2*np.pi,-np.pi,0, np.pi, 2*np.pi],
           [r'$-2\pi$',r'$-\pi$',r'$0$',r'$+\pi$',r'$+2\pi$'])
plt.yticks([-1,0,1,2,3],
           [r'$-1$',r'$0$',r'$+1$',r'$+2$',r'$+3$'])
plt.annotate(r'$\lim_{x\to 0}\frac{\sin(x)}{x}= 1$', xy=[0,1],xycoords='data',
             xytext=[30,30],fontsize=16, textcoords='offset points',
             arrowprops=dict(arrowstyle="->",
             connectionstyle="arc3,rad=.2"))
ax = plt.gca()
ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
ax.xaxis.set_ticks_position('bottom')
ax.spines['bottom'].set_position(('data',0))
ax.yaxis.set_ticks_position('left')
ax.spines['left'].set_position(('data',0))
plt.show()
```

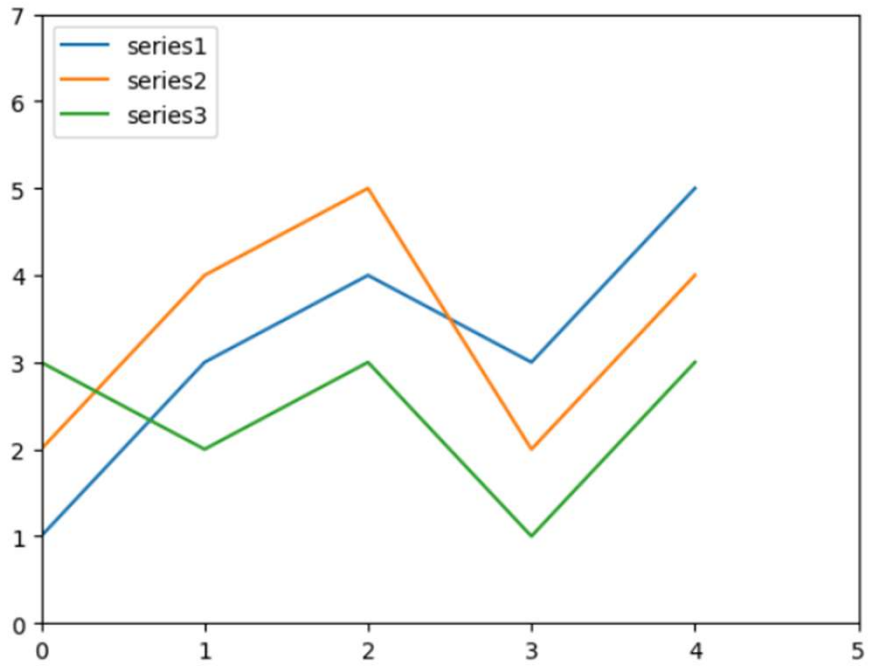


With pandas

- Apply the matplotlib library to the dataframes of the pandas library.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```
data = {'series1':[1,3,4,3,5],
        'series2':[2,4,5,2,4],
        'series3':[3,2,3,1,3]}
df = pd.DataFrame(data)
x = np.arange(5)
plt.axis([0,5,0,7])
plt.plot(x,df)
plt.legend(data, loc=2)
plt.show()
```



Histograms

- A **histogram** is a graphical representation of the **distribution of numerical data**. It shows how data values are grouped into ranges (called *bins*) and how frequently values fall into each range.
- In data analytics, histograms help you quickly understand:
  - Data distribution shape
  - Central tendency (where values concentrate)
  - Spread (variance)
  - Skewness (left/right leaning)
  - Presence of outliers
- This kind of visualization is commonly used in statistical studies about distribution of samples

- **How a Histogram Works**
  - The data range is divided into **bins** (intervals).
  - The number of data points inside each bin is counted.
  - Bars are drawn where:
    - **X-axis** = value ranges (bins)
    - **Y-axis** = frequency (count of values)
- Unlike bar charts, histogram bars **touch each other** because the data is continuous.

- Histograms are used to:
  - Analyze **sales distribution**
  - Evaluate **exam scores**
  - Study **customer age distribution**
  - Detect **anomalies in datasets**
  - Check **normality assumptions** before statistical modeling
- They are often the first step in **Exploratory Data Analysis (EDA)**.

- In Matplotlib's pyplot, the hist() function is used to create histograms.
- Unlike most plotting functions, hist() not only draws the histogram but also **returns a tuple of computed values**.
- The function automatically:
  - Divides the data range into intervals (bins).
  - Counts how many values fall into each bin.
  - Displays the histogram graphically.
  - Returns the calculated results (such as bin counts and bin edges) as a tuple.
- So, hist() performs both the **calculation** and the **visualization** of the histogram.

- Example

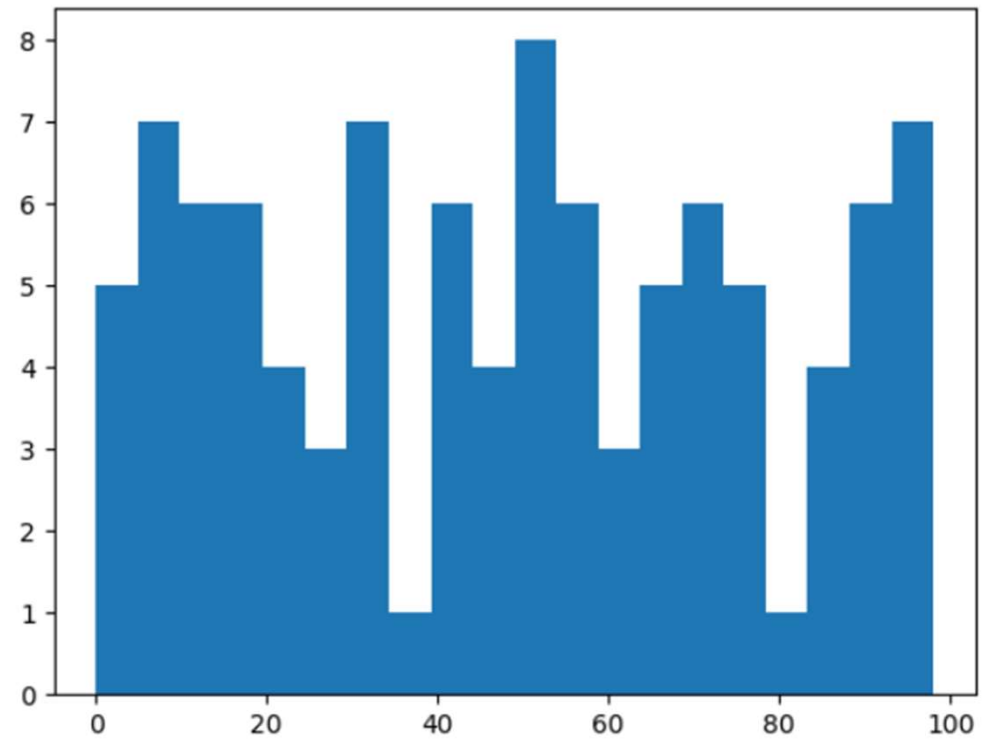
```
import matplotlib.pyplot as plt
import numpy as np
```

```
pop = np.random.randint(0,100,100)
pop
```

```
array([92, 98, 87, 63, 55,  9, 20,  1, 10, 72, 55, 71, 45, 11, 65, 25, 43,
       30, 33, 75, 42, 96, 31, 68, 56, 32, 12,  8, 19, 20, 49, 72, 16,  8,
       52, 52, 33,  9, 50, 23, 12, 84, 36, 53, 10, 44, 12, 71, 42, 57,  8,
        0, 50, 17, 19, 73, 75, 95, 64,  4, 97, 52, 78, 22, 91,  6, 95, 68,
       59, 93, 86, 18, 64, 98, 25, 45, 41, 86, 96, 48, 91, 53,  3, 28,  8,
       81, 42, 58, 91, 18, 33, 55, 31, 76, 46, 70,  1, 60, 74, 89])
```

- Create the histogram of these samples by passing as an argument the hist() function.
  - To divide the occurrences into 20 bins (if not specified, the default value is ten bins)

```
n, bin, patches = plt.hist(pop, bins=20)
plt.show()
```



Bar Charts

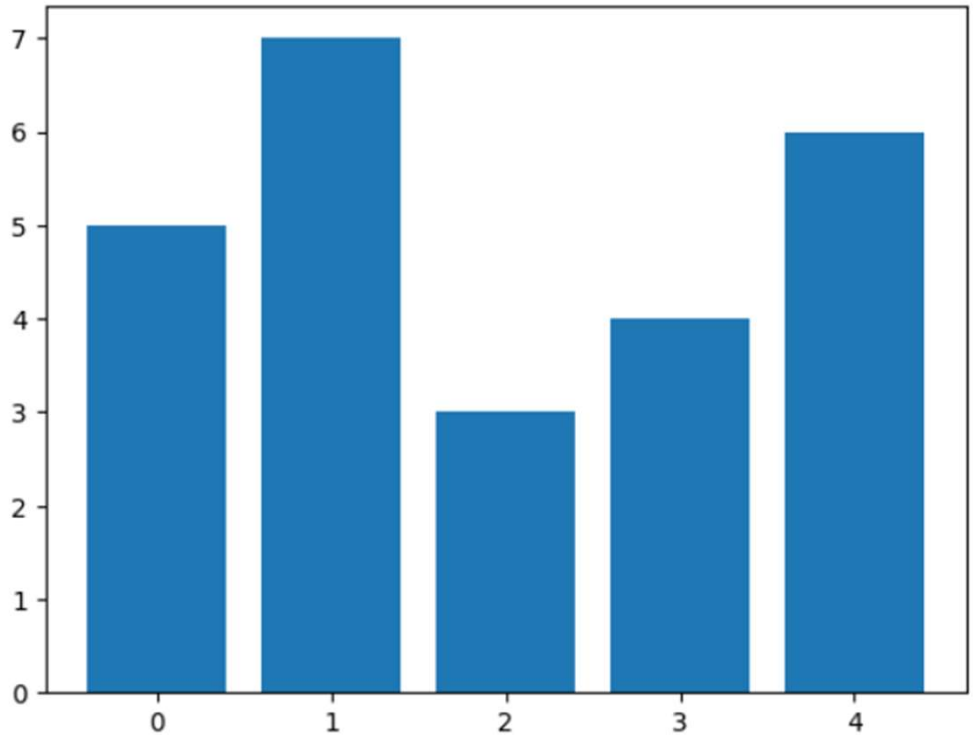
- A **bar chart** is a visualization used to compare **categorical data** by displaying rectangular bars whose lengths represent numerical values.
- In data analytics, bar charts are used to compare quantities across categories clearly and effectively.
- Bar charts are ideal for:
  - Comparing sales across products
  - Comparing revenue by region
  - Showing number of customers by category
  - Displaying survey results
  - Comparing performance metrics
- They are best suited for **discrete (categorical) data**, not continuous distributions (that's what histograms are for).

- **Structure of a Bar Chart**
  - **X-axis** → Categories (e.g., Products, Regions, Departments)
  - **Y-axis** → Numerical values (e.g., Sales, Count, Profit)
- Bars are **separated**, unlike histograms
- **Types of Bar Charts**
  - **Vertical bar chart** → bar()
  - **Horizontal bar chart** → barh()
  - **Grouped bar chart** → Compare multiple categories
  - **Stacked bar chart** → Show composition within categories

- Bar charts help analysts:
  - Compare categories quickly
  - Identify highest/lowest performers
  - Communicate business insights clearly
  - Support decision-making
- They are widely used in dashboards, business intelligence reports, and presentations.

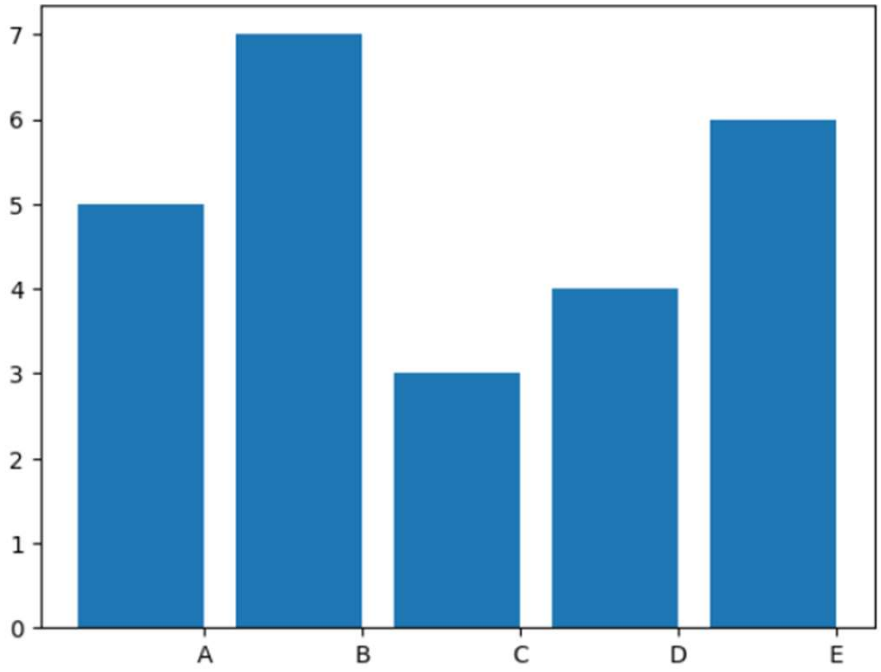
- The realization of the bar chart is very simple with matplotlib, using the **bar()** function.

```
import matplotlib.pyplot as plt

index = [0,1,2,3,4]
values = [5,7,3,4,6]
plt.bar(index,values)
plt.show()
```



- list containing the values corresponding to their positions on the **x-axis** as the first argument of the **xticks()** function.

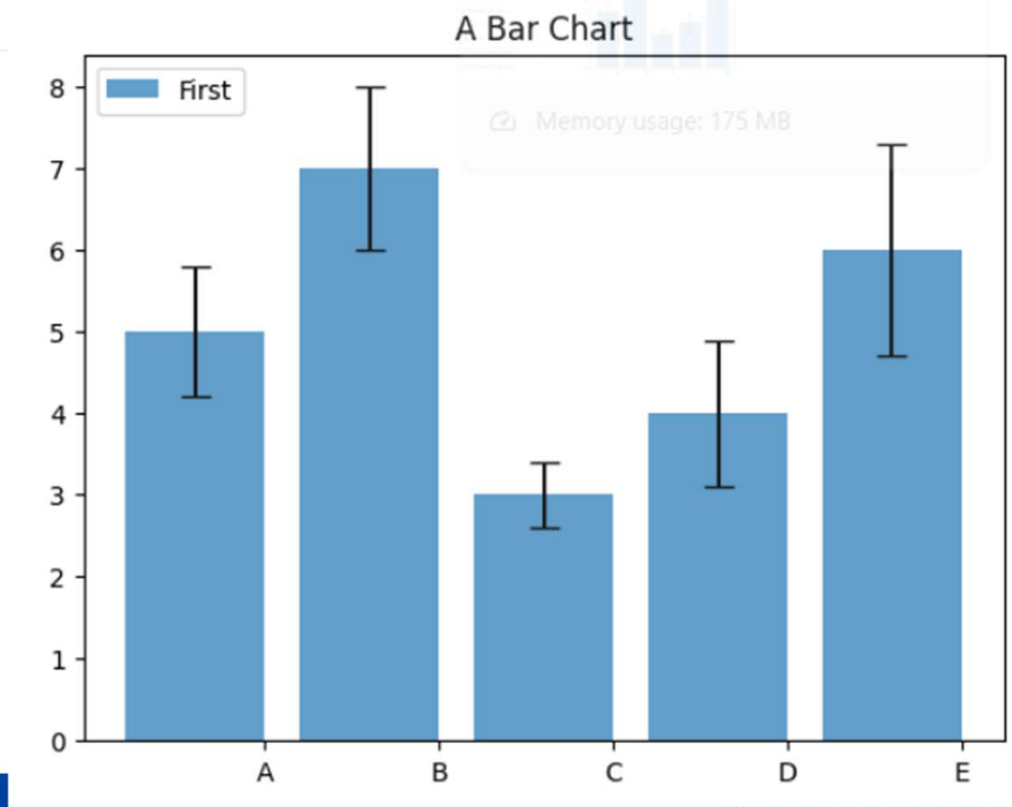
```
import matplotlib.pyplot as plt
import numpy as np

index = np.arange(5)
values1 = [5,7,3,4,6]
plt.bar(index, values1)
plt.xticks(index+0.4,['A','B','C','D','E'])
plt.show()
```



A simple bar chart with categories on the x-axis

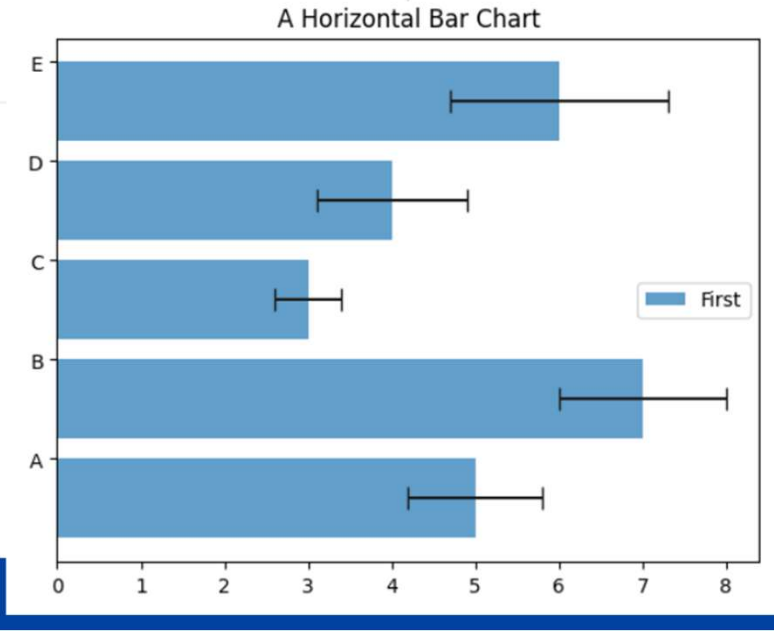
- Bar charts in Matplotlib can be further refined by adding specific keyword arguments (kwargs) to the bar() function.
- **Key Enhancements:**
- **Error bars**:
  - yerr → Adds standard deviation or error values (as a list).
  - error_kw → Customizes error bar appearance.
    - ecolor → Sets the color of the error bars.
    - capsize → Defines the width of the small horizontal lines at the ends of error bars.
- **Transparency**:
  - alpha → Controls bar transparency.
    - Range: 0 (fully transparent) to 1 (fully opaque).
- **Legend labeling**:
  - label → Identifies the data series for use in a legend.

```
index = np.arange(5)
values1 = [5,7,3,4,6]
std1 = [0.8,1,0.4,0.9,1.3]
plt.title('A Bar Chart')
plt.bar(index, values1, yerr=std1, error_kw={'ecolor':'0.1','capsize':6},alpha=0.7,label='First')
plt.xticks(index+0.4,['A','B','C','D','E'])
plt.legend(loc=2)
plt.show()
```



Horizontal Bar Chart: barh() function

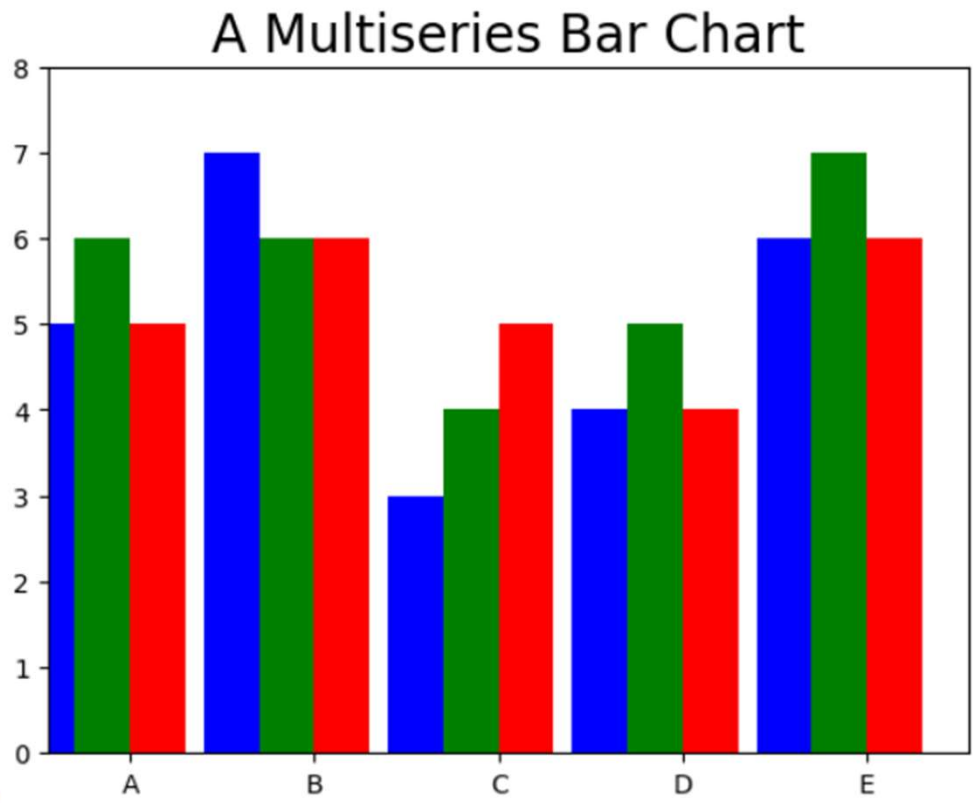
```
index = np.arange(5)
values1 = [5,7,3,4,6]
std1 = [0.8,1,0.4,0.9,1.3]
plt.title('A Horizontal Bar Chart')
plt.barh(index, values1, xerr=std1, error_kw={'ecolor':'0.1','capsize':6},alpha=0.7,label='First')
plt.yticks(index+0.4,['A','B','C','D','E'])
plt.legend(loc=5)
plt.show()
```



ng sự khác biệt

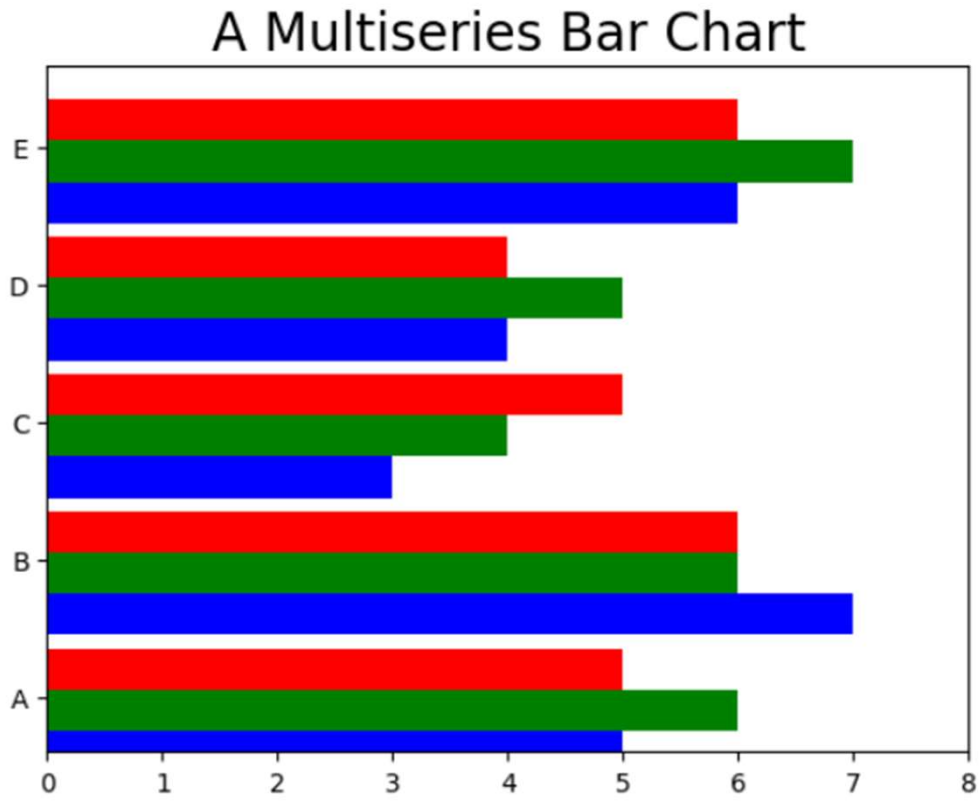
Multiserial Bar Chart: xticks() function

```
#Multiserial Bar Chart
index = np.arange(5)
values1 = [5,7,3,4,6]
values2 = [6,6,4,5,7]
values3 = [5,6,5,4,6]
bw = 0.3
plt.axis([0,5,0,8])
plt.title('A Multiseries Bar Chart', fontsize=20)
plt.bar(index, values1, bw, color='b')
plt.bar(index+bw, values2, bw, color='g')
plt.bar(index+2*bw, values3, bw, color='r')
plt.xticks(index+1.5*bw,['A','B','C','D','E'])
plt.show()
```



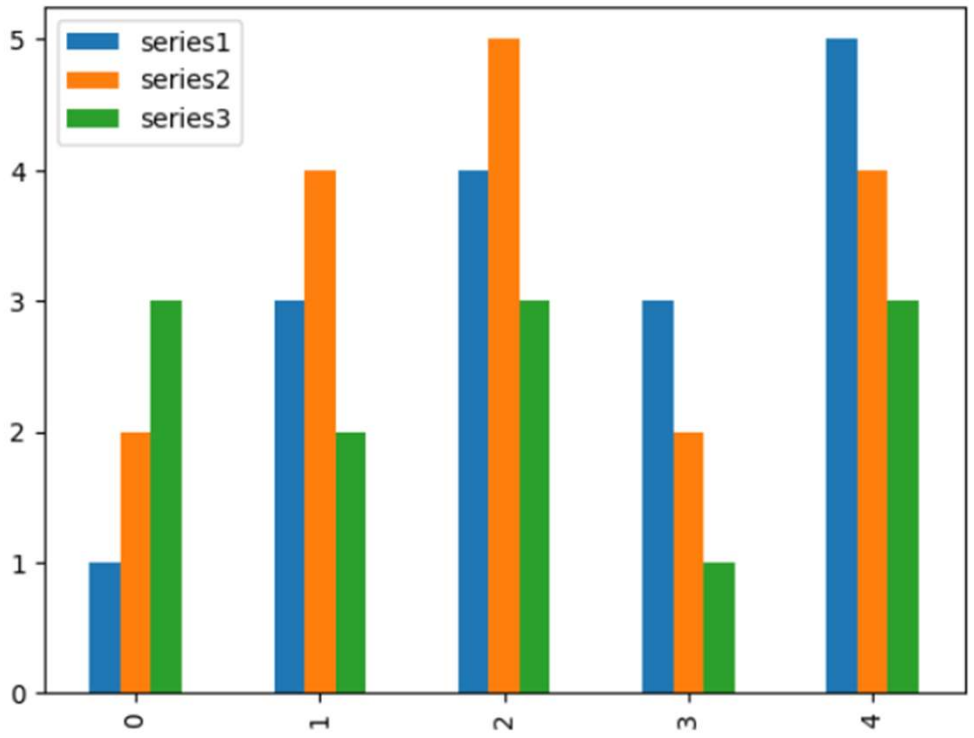
- **Multiserial Bar Chart:** yticks() function

```
#Multiserial Bar Chart
index = np.arange(5)
values1 = [5,7,3,4,6]
values2 = [6,6,4,5,7]
values3 = [5,6,5,4,6]
bw = 0.3
plt.axis([0,8,0,5])
plt.title('A Multiseries Bar Chart', fontsize=20)
plt.barh(index, values1, bw, color='b')
plt.barh(index+bw, values2, bw, color='g')
plt.barh(index+2*bw, values3, bw, color='r')
plt.yticks(index+0.4,['A','B','C','D','E'])
plt.show()
```

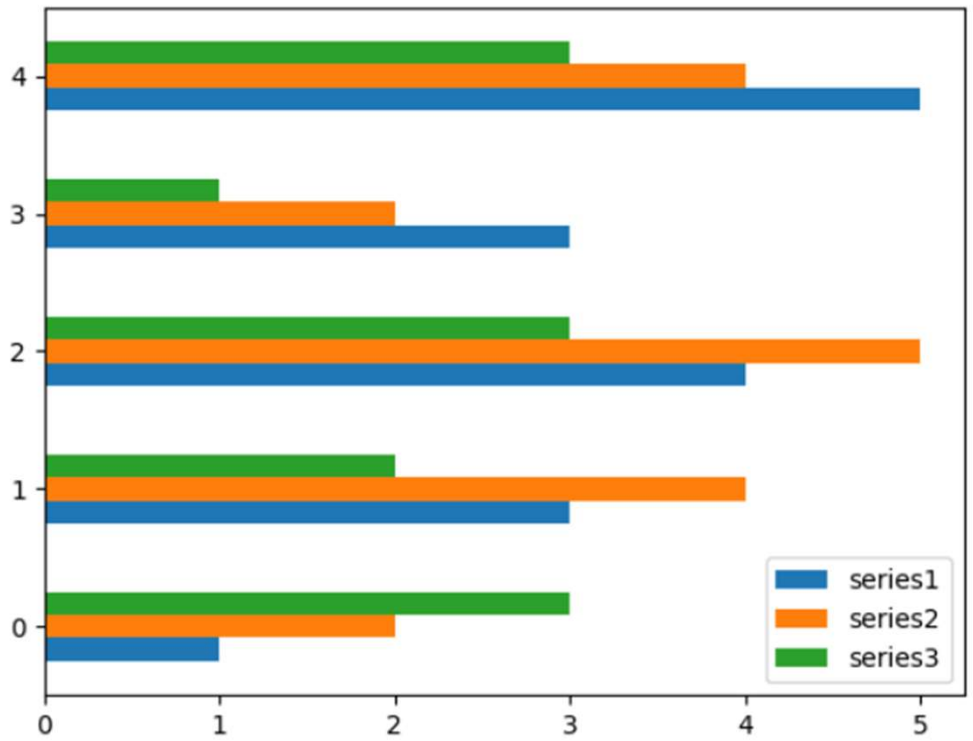


Multiseries Bar Charts with a pandas Dataframe

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

index = np.arange(5)
data = {'series1': [1,3,4,3,5],
        'series2': [2,4,5,2,4],
        'series3': [3,2,3,1,3]}
df = pd.DataFrame(data)
df.plot(kind='bar')
plt.show()
```

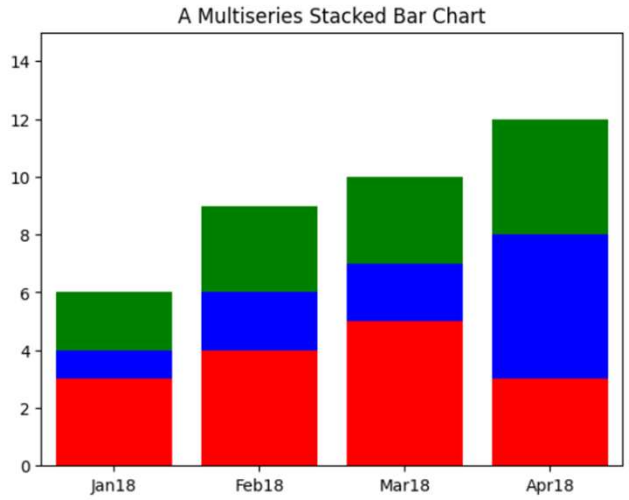


```
index = np.arange(5)
data = {'series1': [1,3,4,3,5],
        'series2': [2,4,5,2,4],
        'series3': [3,2,3,1,3]}
df = pd.DataFrame(data)
df.plot(kind='barh')
plt.show()
```

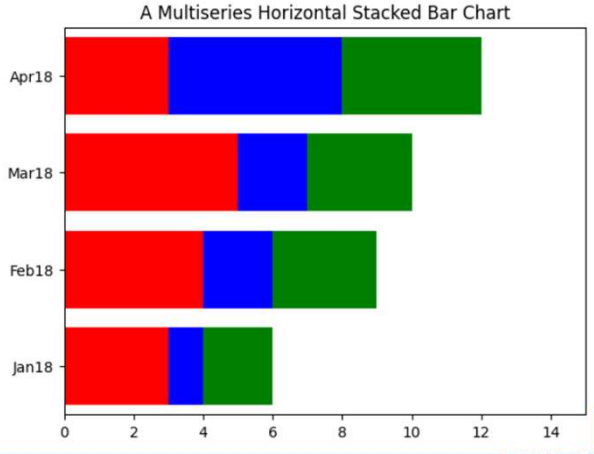


Multiseries Stacked Bar Charts

```
#Multiseries Stacked Bar Charts
series1 = np.array([3,4,5,3])
series2 = np.array([1,2,2,5])
series3 = np.array([2,3,3,4])
index = np.arange(4)
plt.axis([-0.5,3.5,0,15])
plt.title('A Multiseries Stacked Bar Chart')
plt.bar(index,series1,color='r')
plt.bar(index,series2,color='b',bottom=series1)
plt.bar(index,series3,color='g',bottom=(series2+series1))
plt.xticks(index,['Jan18','Feb18','Mar18','Apr18'])
plt.show()
```

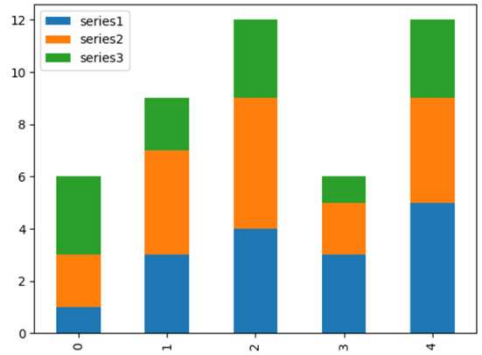



```
series1 = np.array([3,4,5,3])
series2 = np.array([1,2,2,5])
series3 = np.array([2,3,3,4])
index = np.arange(4)
plt.axis([0,15,-0.5,3.5])
plt.title('A Multiseries Horizontal Stacked Bar Chart')
plt.barh(index,series1,color='r')
plt.barh(index,series2,color='b',left=series1)
plt.barh(index,series3,color='g',left=(series2+series1))
plt.yticks(index,['Jan18','Feb18','Mar18','Apr18'])
plt.show()
```

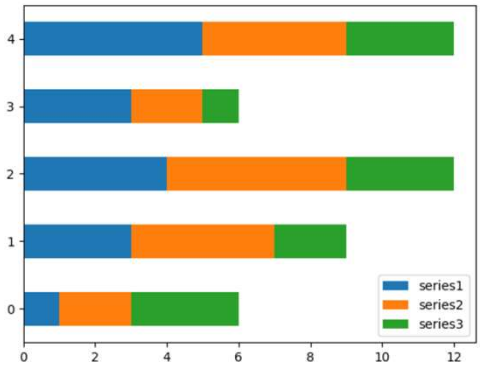


- # Stacked Bar Charts with a pandas Dataframe

```
#Stacked Bar Charts with a pandas Dataframe
data = {'series1': [1,3,4,3,5],
        'series2': [2,4,5,2,4],
        'series3': [3,2,3,1,3]}
df = pd.DataFrame(data)
df.plot(kind='bar',stacked=True)
plt.show()
```

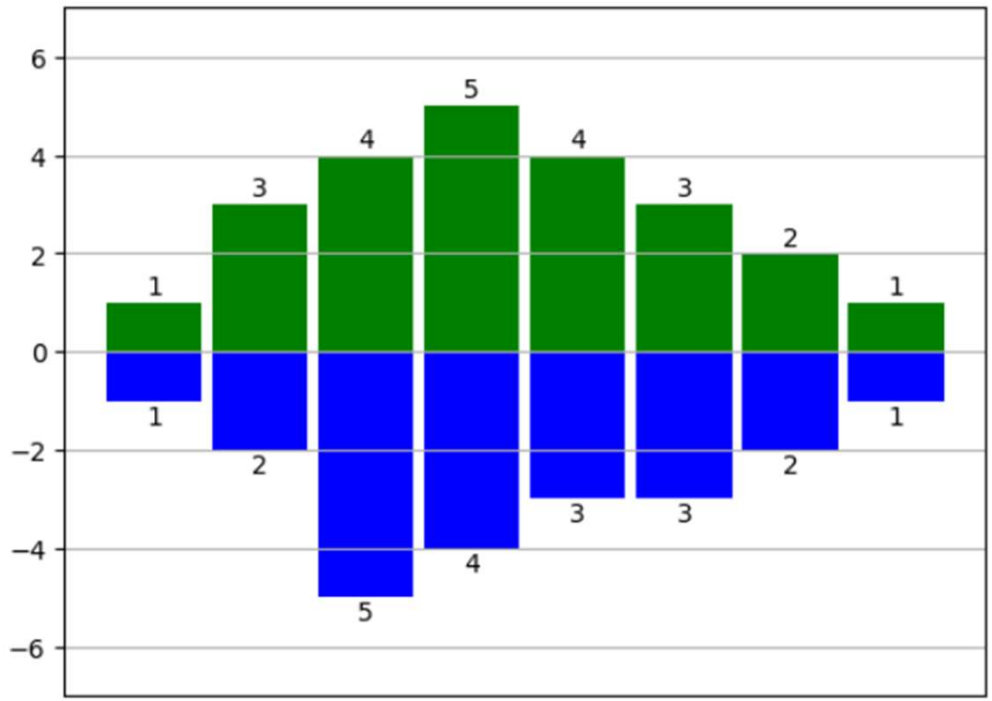



```
#Stacked Bar Charts with a pandas Dataframe
data = {'series1': [1,3,4,3,5],
        'series2': [2,4,5,2,4],
        'series3': [3,2,3,1,3]}
df = pd.DataFrame(data)
df.plot(kind='barh',stacked=True)
plt.show()
```



Other Bar Chart Representations

```
#Other Bar Chart Representation
x0 = np.arange(8)
y1 = np.array([1,3,4,5,4,3,2,1])
y2 = np.array([1,2,5,4,3,3,2,1])
plt.ylim(-7,7)
plt.bar(x0,y1,0.9, facecolor='g')
plt.bar(x0,-y2,0.9,facecolor='b')
plt.xticks(())
plt.grid(True)
for x, y in zip(x0, y1):
    plt.text(x, y + 0.05, '%d' % y, ha='center', va = 'bottom')
for x, y in zip(x0, y2):
    plt.text(x, -y - 0.05, '%d' % y, ha='center', va = 'top')
plt.show()
```



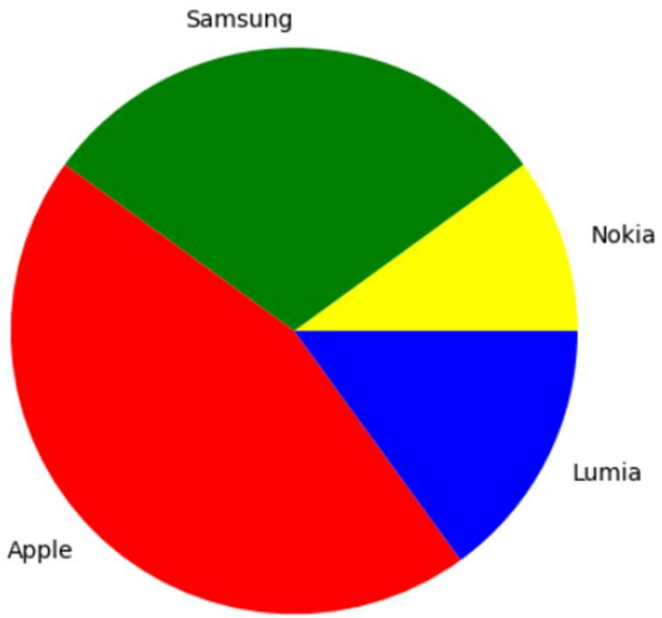
Pie Charts

- A **pie chart** is a circular chart used to show **proportions or percentages** of a whole. Each slice of the pie represents a category's contribution relative to the total.
- Pie charts are commonly used when you want to visualize **part-to-whole relationships**.
- Pie charts are useful for:
  - Market share distribution
  - Budget allocation
  - Survey response percentages
  - Customer segmentation
  - Product category contribution
- They work best when:
  - There are **few categories** (typically 3–6).
  - The goal is to emphasize **percentage contribution**.

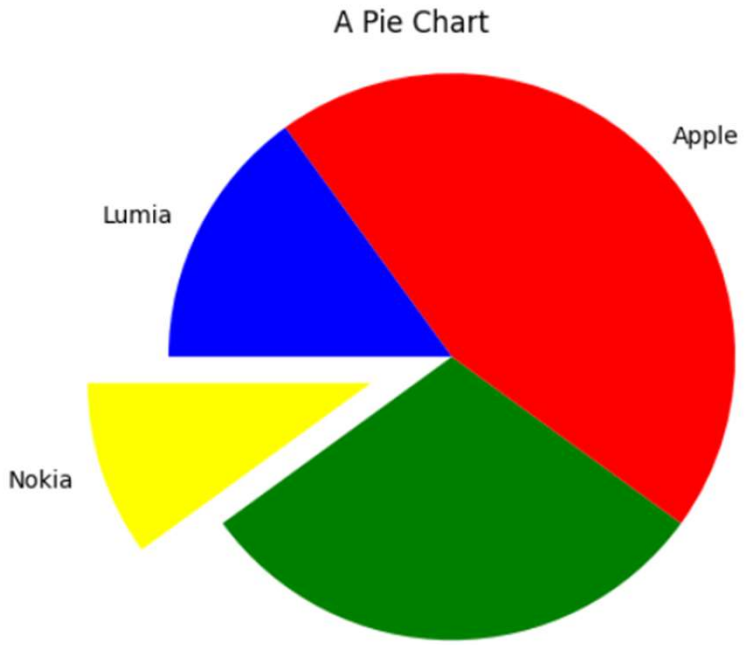
- **Structure of a Pie Chart**
  - The entire circle represents **100%**.
  - Each slice represents a category.
  - Slice size is proportional to its value.
- **Common Parameters:**
  - labels → Category names
  - autopct → Displays percentages
  - startangle → Rotates the chart
  - explode → Separates a slice for emphasis
  - shadow → Adds shadow effect

- **Advantages in Data Analytics**
  - Communicate proportions quickly
  - Highlight dominant categories
  - Present simple business summaries
  - Support executive-level reporting
- **Limitations**
  - Not suitable for many categories
  - Hard to compare small differences
  - Less precise than bar charts

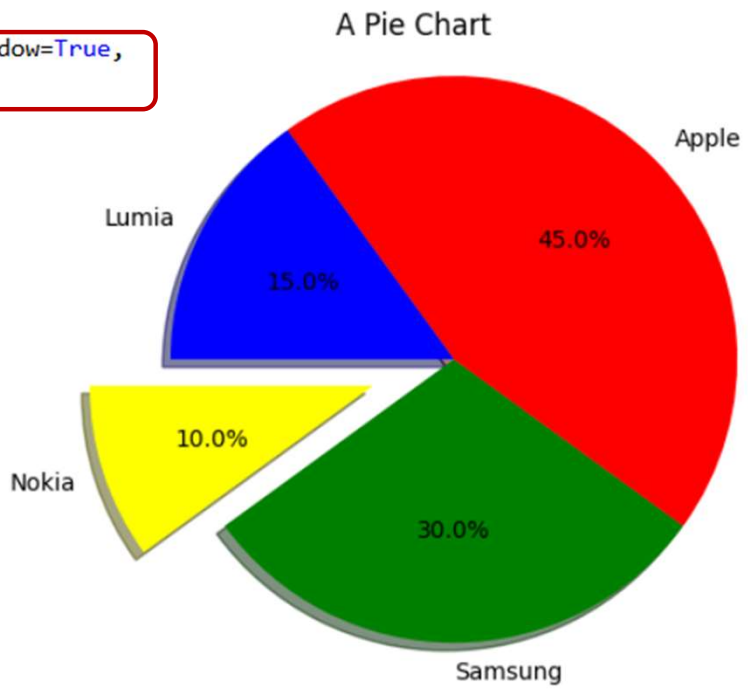
```
labels = ['Nokia','Samsung','Apple','Lumia']
values = [10,30,45,15]
colors = ['yellow','green','red','blue']
plt.pie(values,labels=labels,colors=colors)
plt.axis('equal')
plt.show()
```




```
labels = ['Nokia','Samsung','Apple','Lumia']
values = [10,30,45,15]
colors = ['yellow','green','red','blue']
explode = [0.3,0,0,0]
plt.title('A Pie Chart')
plt.pie(values,labels=labels,colors=colors,explode=explode,startangle=180)
plt.axis('equal')
plt.show()
```

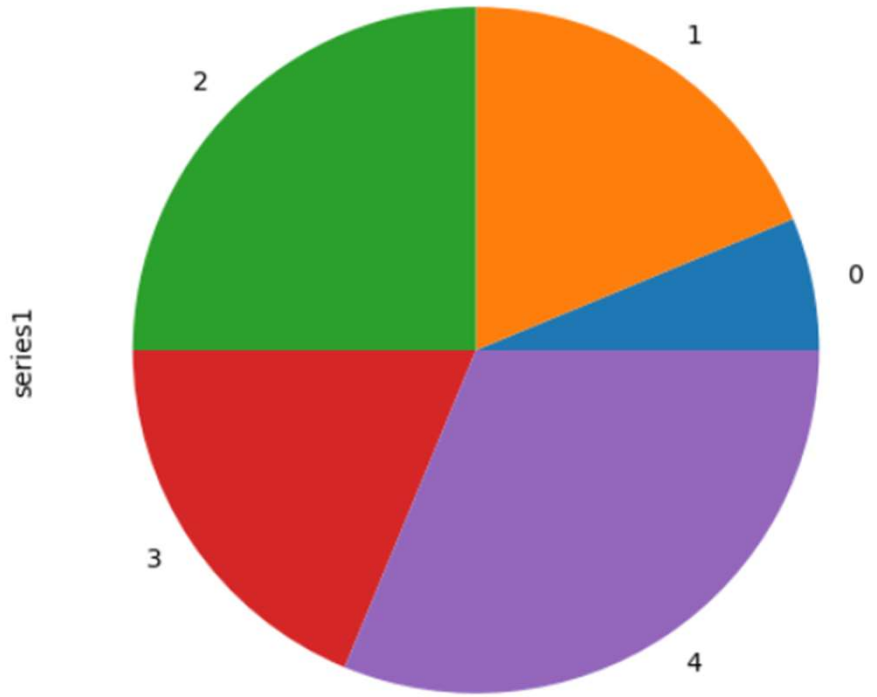


```
labels = ['Nokia','Samsung','Apple','Lumia']
values = [10,30,45,15]
colors = ['yellow','green','red','blue']
explode = [0.3,0,0,0]
plt.title('A Pie Chart')
plt.pie(values,labels=labels,colors=colors,explode=explode,shadow=True,
        autopct='%1.1f%%',startangle=180)
plt.axis('equal')
plt.show()
```



Pie Charts with a pandas Dataframe

```
#Pie Charts with a pandas Dataframe
data = {'series1': [1,3,4,3,5],
        'series2': [2,4,5,2,4],
        'series3': [3,2,3,1,3]}
df = pd.DataFrame(data)
df['series1'].plot(kind='pie', figsize=(6,6))
plt.show()
```

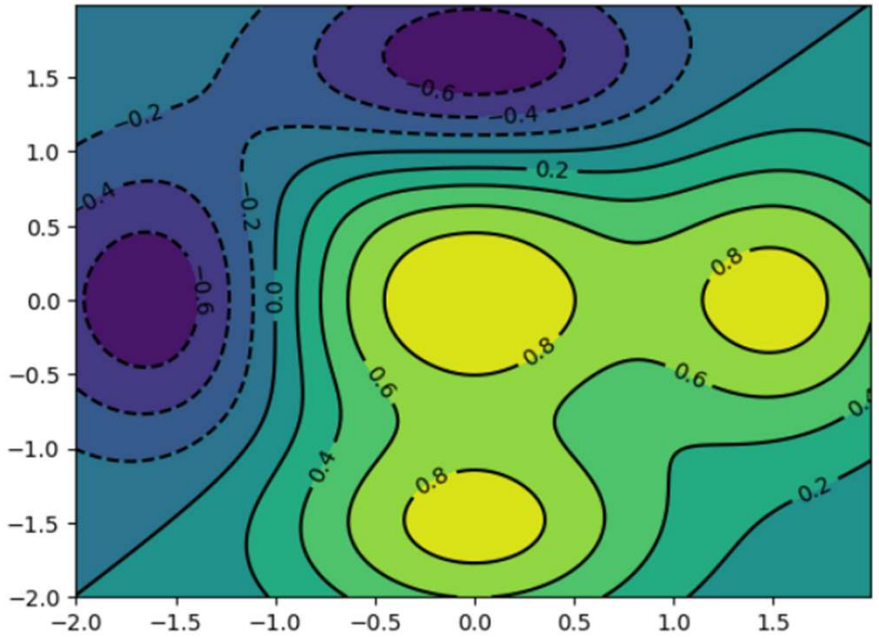


Tôn trọng sự khác biệt

Advanced Charts

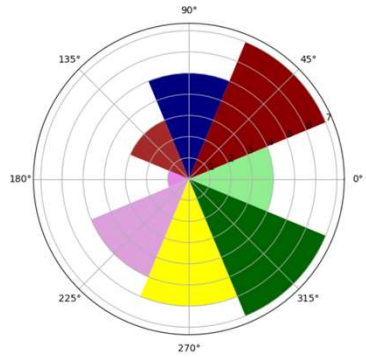
- **Contour Plots:** A **contour plot** is a graphical representation of a **three-dimensional surface** on a two-dimensional plane. It shows how a variable Zchanges based on two independent variables $X$and Y.

```
dx = 0.01; dy = 0.01
x = np.arange(-2.0,2.0,dx)
y = np.arange(-2.0,2.0,dy)
X,Y = np.meshgrid(x,y)

def f(x,y):
    return (1 - y**5 + x**5)*np.exp(-x**2-y**2)

C = plt.contour(X,Y,f(X,Y),8,colors='black')
plt.contourf(X,Y,f(X,Y),8)
plt.clabel(C, inline=1, fontsize=10)
plt.show()
```



- **Polar Charts**: A **polar chart** is a type of plot that represents data using **polar coordinates** instead of Cartesian coordinates.
- In a polar coordinate system:
- Each point is defined by:
  - **Radius (r)** → Distance from the center
  - **Angle (θ)** → Direction from the reference axis
- Instead of using x and y axes, polar charts use circular geometry.

```
N = 8
theta = np.arange(0.,2 * np.pi, 2 * np.pi / N)
radii = np.array([4,7,5,3,1,5,6,7])
plt.axes([0.025, 0.025, 0.95, 0.95], polar=True)
colors = np.array(['lightgreen','darkred','navy','brown',
                   'violet','plum','yellow','darkgreen'])
bars = plt.bar(theta, radii, width=(2*np.pi/N), bottom=0.0, color=colors)
plt.show()
```



mplot3d

- mplot3d is a **3D plotting toolkit** in Matplotlib that allows you to create three-dimensional visualizations in Python.
- It extends Matplotlib's standard 2D plotting capabilities to support 3D graphs.
- With mplot3d, you can create:
  - 3D line plots
  - 3D scatter plots
  - 3D surface plots
  - Wireframe plots
  - 3D contour plots
  - 3D bar charts
- It enables visualization of relationships among **three variables (X, Y, Z)**.

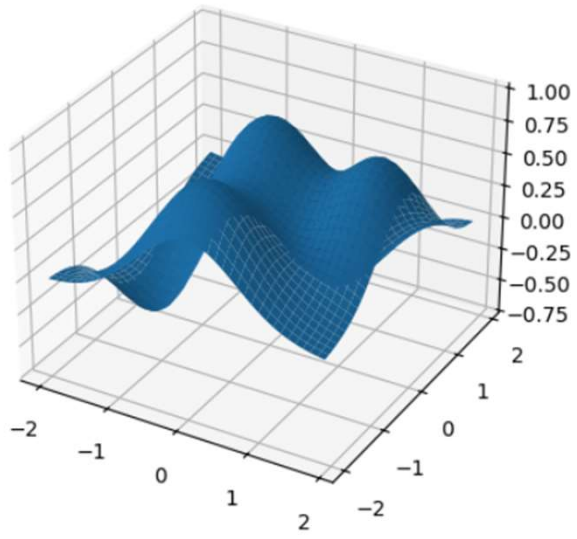
- It is useful for:
  - Visualizing mathematical functions
  - Scientific simulations
  - Engineering models
  - Optimization surfaces
  - Multivariable data analysis

3D Surfaces

- view the surface with the **plot_surface()** function

```
#3D Surfaces
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
```

```
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
X = np.arange(-2,2,0.1)
Y = np.arange(-2,2,0.1)
X,Y = np.meshgrid(X,Y)

def f(x,y):
    return (1 - y**5 + x**5)*np.exp(-x**2-y**2)

ax.plot_surface(X,Y,f(X,Y), rstride=1, cstride=1)
plt.show()
```



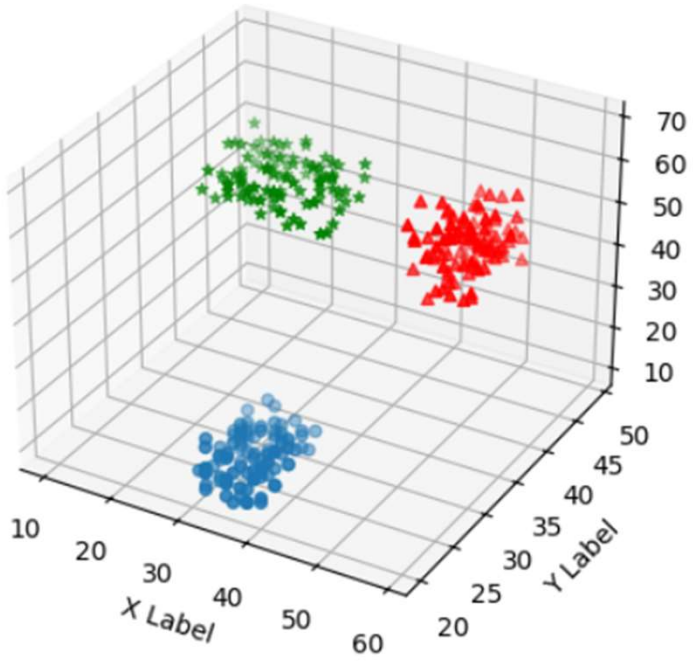
3D scatter plot

- A **3D scatter plot** is a visualization that displays data points in **three-dimensional space**, where each point is defined by three variables:
  - **X-axis** → First variable
  - **Y-axis** → Second variable
  - **Z-axis** → Third variable
- Each point represents one observation in a dataset.
- 3D scatter plots are useful when:
  - You want to analyze relationships among **three numerical variables**
  - You need to detect **clusters or patterns**
  - You want to identify **outliers**
  - You are visualizing multidimensional datasets
- They are common in:
  - Scientific research
  - Engineering
  - Machine learning
  - Data analytics

- **Advantages**
- Visualizes three variables simultaneously
- Reveals spatial patterns
- Helps detect clusters and anomalies

- **Limitations**
- Can become cluttered with many points
- Perspective distortion may affect interpretation
- Harder to interpret than 2D plots

```python
#Scatter Plots in 3D
from mpl_toolkits.mplot3d import Axes3D

xs = np.random.randint(30,40,100)
ys = np.random.randint(20,30,100)
zs = np.random.randint(10,20,100)
xs2 = np.random.randint(50,60,100)
ys2 = np.random.randint(30,40,100)
zs2 = np.random.randint(50,70,100)
xs3 = np.random.randint(10,30,100)
ys3 = np.random.randint(40,50,100)
zs3 = np.random.randint(40,50,100)

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.scatter(xs,ys,zs)
ax.scatter(xs2,ys2,zs2,c='r',marker='^')
ax.scatter(xs3,ys3,zs3,c='g',marker='*')
ax.set_xlabel('X Label')
ax.set_ylabel('Y Label')
ax.set_xlabel('X Label')
plt.show()
```



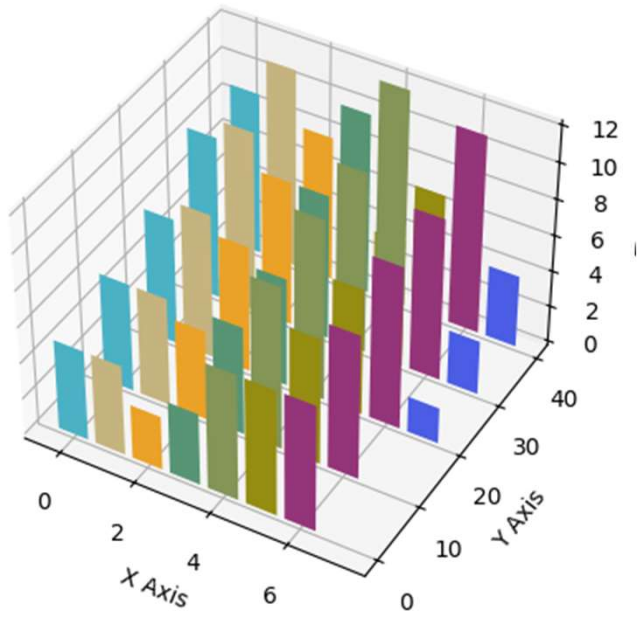
Bar Charts in 3D

- A **3D bar chart** is a three-dimensional extension of a traditional bar chart. It represents data using rectangular bars that extend along the **Z-axis**, while positioned across the **X and Y axes**.
- In other words:
  - **X-axis** → First categorical variable
  - **Y-axis** → Second categorical variable (or grouping variable)
  - **Z-axis (height)** → Numerical value
- 3D bar charts are useful when:
  - Comparing values across **two categorical dimensions**
  - Visualizing matrix-like data
  - Showing grouped category comparisons
  - Displaying multi-dimensional business metrics
- Example use cases:
  - Sales by **Product and Region**
  - Revenue by **Month and Department**
  - Performance by **Category and Year**

- **Advantages**
- Displays three dimensions simultaneously
- Good for matrix-style data
- Visually impressive for presentations

- **Limitations**
- Can be harder to interpret due to perspective
- May distort perception of heights
- Often less clear than 2D grouped bar charts

```python
#Bar Chart 3D
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D
x = np.arange(8)
y = np.random.randint(0,10,8)
y2 = y + np.random.randint(0,3,8)
y3 = y2 + np.random.randint(0,3,8)
y4 = y3 + np.random.randint(0,3,8)
y5 = y4 + np.random.randint(0,3,8)
clr = ['#4bb2c5', '#c5b47f', '#EAA228', '#579575',
       '#839557', '#958c12', '#953579', '#4b5de4']
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.bar(x,y,0,zdir='y',color=clr)
ax.bar(x,y2,10,zdir='y',color=clr)
ax.bar(x,y3,20,zdir='y',color=clr)
ax.bar(x,y4,30,zdir='y',color=clr)
ax.bar(x,y5,40,zdir='y',color=clr)
ax.set_xlabel('X Axis')
ax.set_ylabel('Y Axis')
ax.set_zlabel('Z Axis')
ax.view_init(elev=40)
```

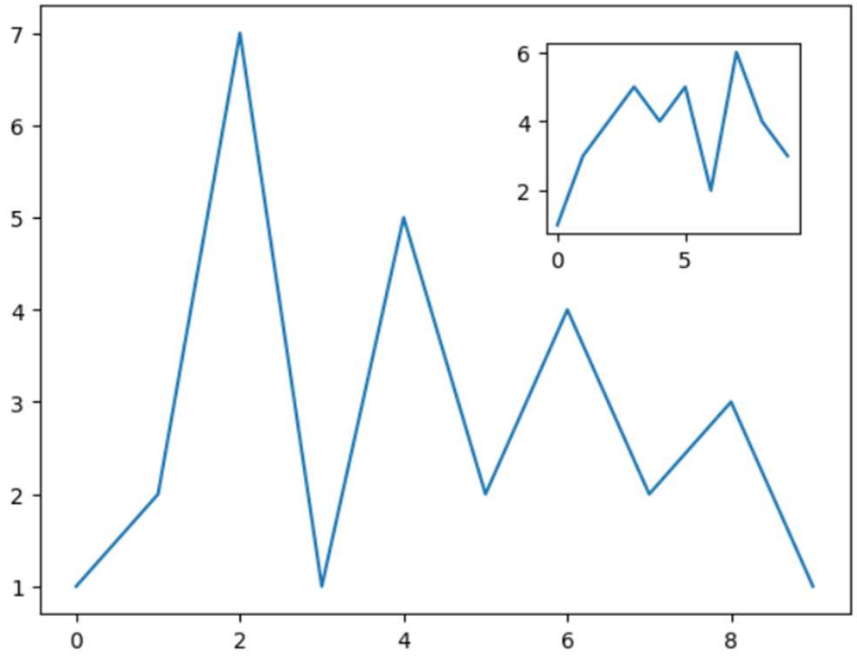


Multipanel Plots

- More charts in the same figure by separating them with subplots

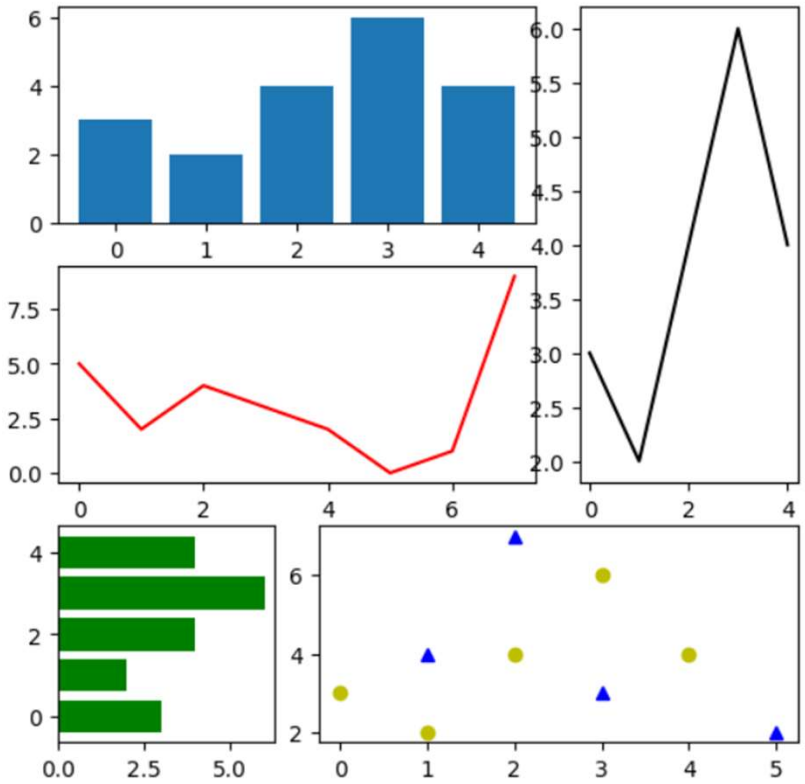
Display Subplots Within Other Subplots

```
fig = plt.figure()
ax = fig.add_axes([0.1,0.1,0.8,0.8])
inner_ax = fig.add_axes([0.6,0.6,0.25,0.25])
x1 = np.arange(10)
y1 = np.array([1,2,7,1,5,2,4,2,3,1])
x2 = np.arange(10)
y2 = np.array([1,3,4,5,4,5,2,6,4,3])
ax.plot(x1,y1)
inner_ax.plot(x2,y2)
plt.show()
```



Grids of Subplots

```
gs = plt.GridSpec(3,3)
fig = plt.figure(figsize=(6,6))
x1 = np.array([1,3,2,5])
y1 = np.array([4,3,7,2])
x2 = np.arange(5)
y2 = np.array([3,2,4,6,4])
s1 = fig.add_subplot(gs[1,:2])
s1.plot(x,y,'r')
s2 = fig.add_subplot(gs[0,:2])
s2.bar(x2,y2)
s3 = fig.add_subplot(gs[2,0])
s3.barh(x2,y2,color='g')
s4 = fig.add_subplot(gs[:2,2])
s4.plot(x2,y2,'k')
s5 = fig.add_subplot(gs[2,1:])
s5.plot(x1,y1,'b^',x2,y2,'yo')
plt.show()
```



The Seaborn Library

- **Seaborn** is a high-level Python data visualization library built on top of Matplotlib. It provides a more attractive, modern, and statistically oriented interface for creating informative graphics.
- It is especially popular in **data analytics, statistics, and exploratory data analysis (EDA)**.
- Key advantages:
  - Beautiful default themes
  - Built-in statistical plots
  - Easy handling of Pandas DataFrames
  - Automatic aggregation and confidence intervals
  - Simple syntax for complex plots

- Seaborn supports:
  - Histograms and distribution plots
  - Box plots and violin plots
  - Bar plots with confidence intervals
  - Scatter plots and regression plots
  - Heatmaps
  - Pair plots (multi-variable relationships)
  - Correlation matrices

- **Seaborn vs Matplotlib**

Feature	Matplotlib	Seaborn
Level	Low-level control	High-level interface
Aesthetics	Manual styling	Built-in modern themes
Statistical plots	Basic	Advanced statistical focus
Ease of use	More code	Less code

- Import seaborn

```
import seaborn as sns
```

- To chart with Seaborn
  - Selecting a theme: set_theme() function
  - Loading a dataset: load_dataset() function
  - Creating the graph: relplot() function

```
#1. set_theme
sns.set_theme(style='darkgrid', palette='Set2')
```

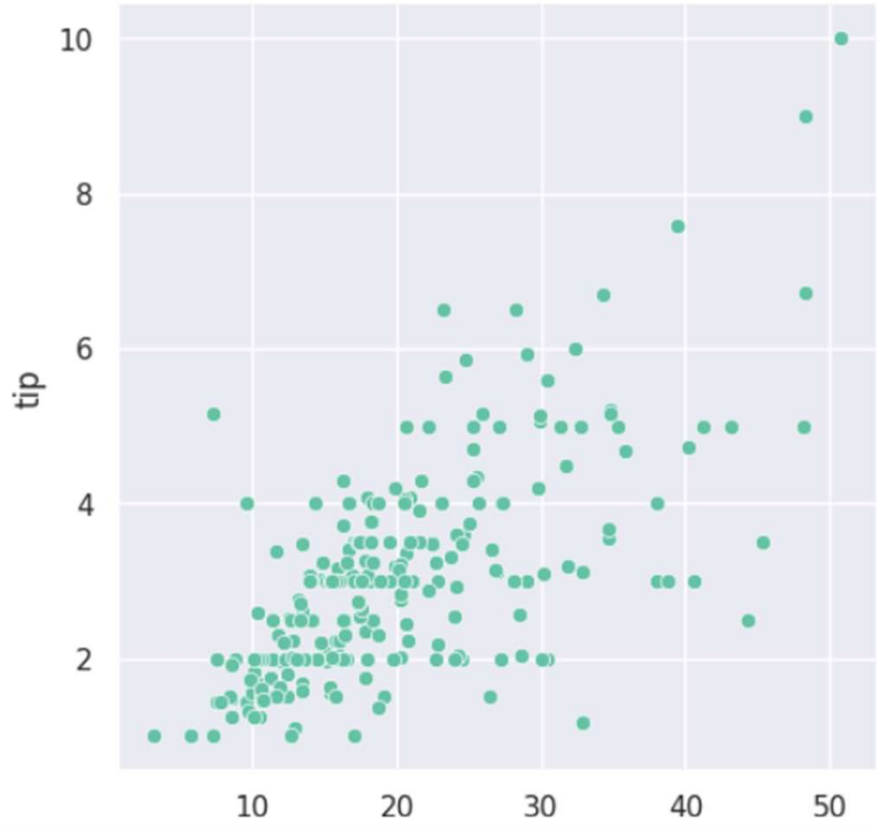
```
#2. load_dataset
tips = sns.load_dataset("tips")
tips
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

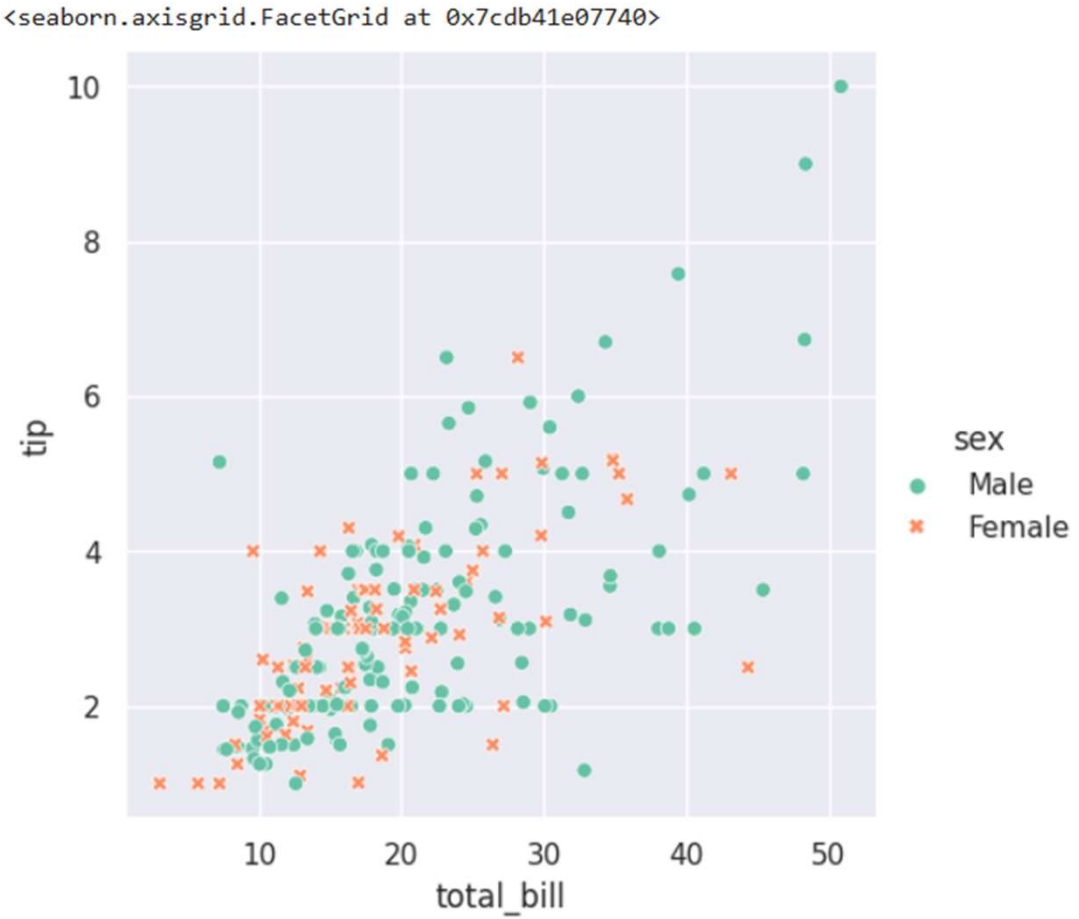
```
#3.
sns.relplot(
data=tips,
x='total_bill', y='tip',
)
```

```
<seaborn.axisgrid.FacetGrid at 0x7cdb41f67e60>
```



- Scatter plot in which the dots separate the data relating to the payer's gender

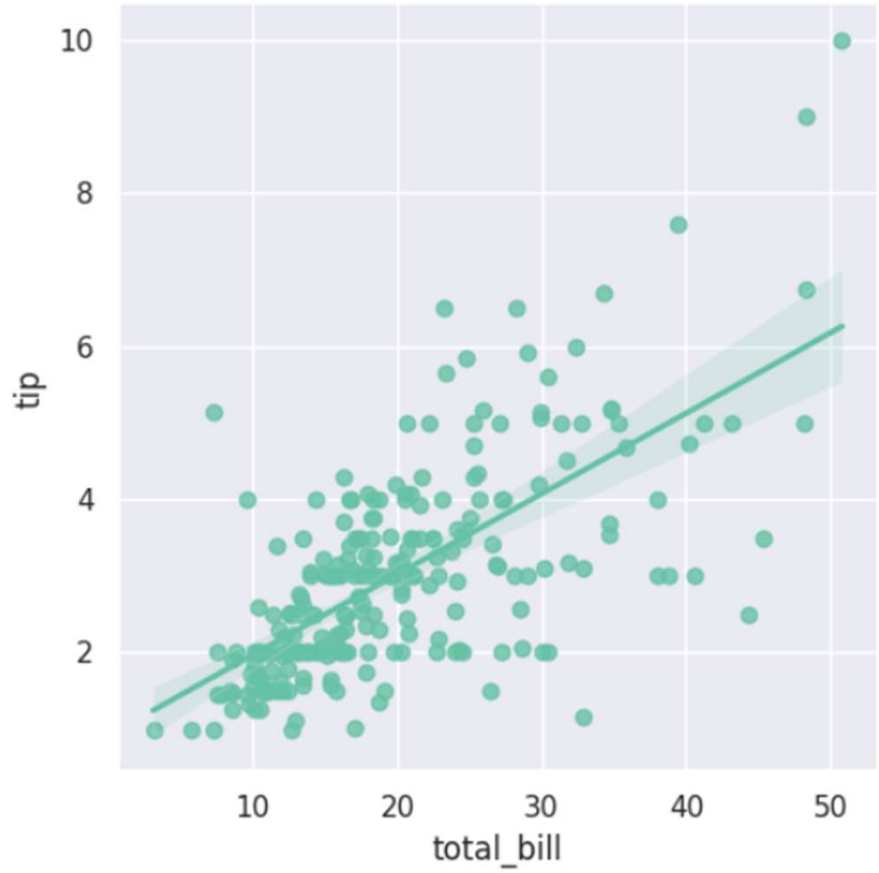
```
sns.relplot(
data=tips,
x='total_bill', y='tip',
hue='sex', style='sex',
)
```



- Scatter plot with a linear regression graphic element

```
sns.lmplot(data=tips, x='total_bill', y='tip')
```

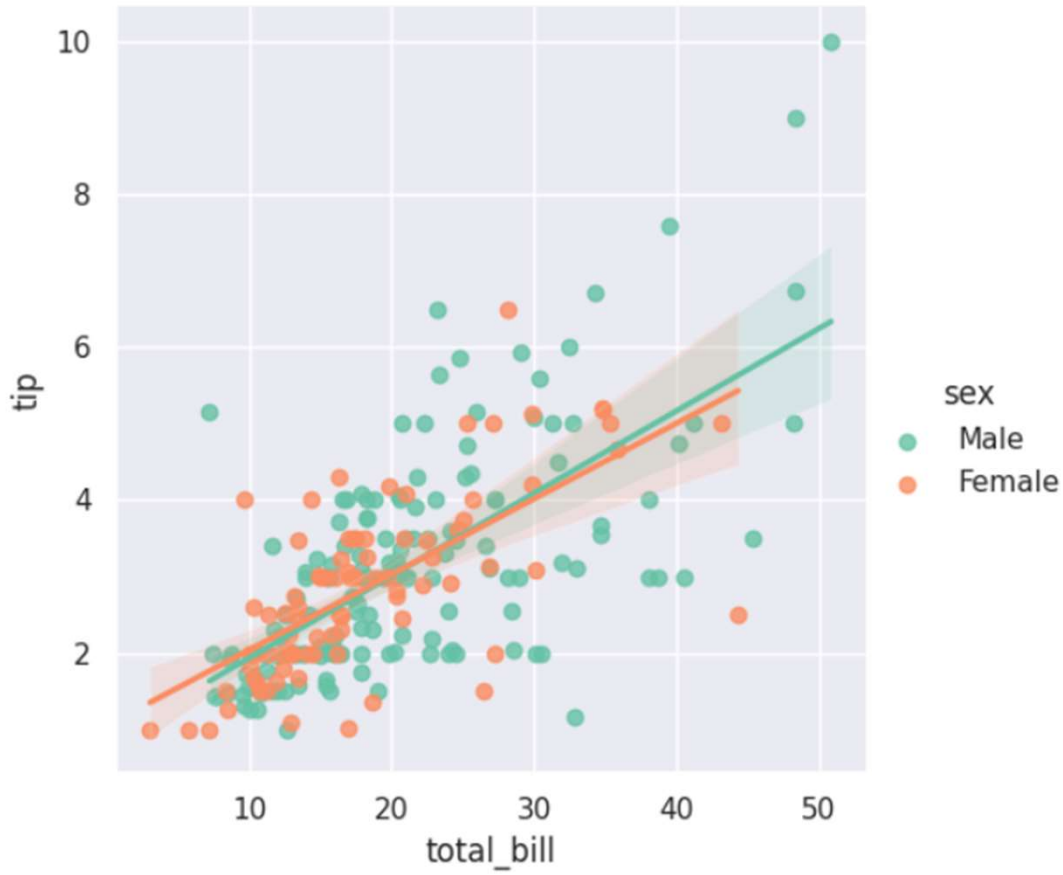
```
<seaborn.axisgrid.FacetGrid at 0x7cdb41e1e3f0>
```



- The scatter plot with linear regressions performed by gender

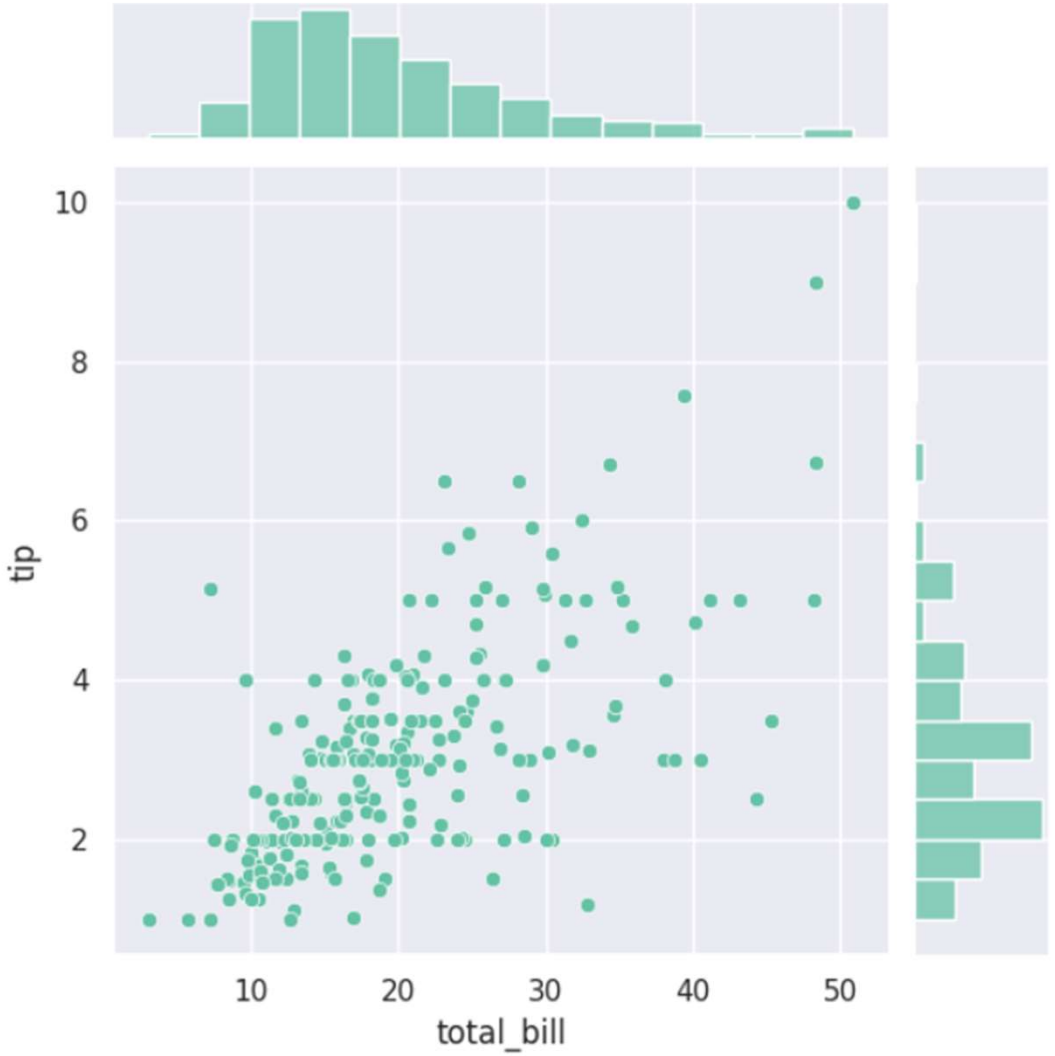
```
sns.lmplot(data=tips, x='total_bill', y='tip', hue='sex')
```

```
<seaborn.axisgrid.FacetGrid at 0x7cdb41ff6ab0>
```



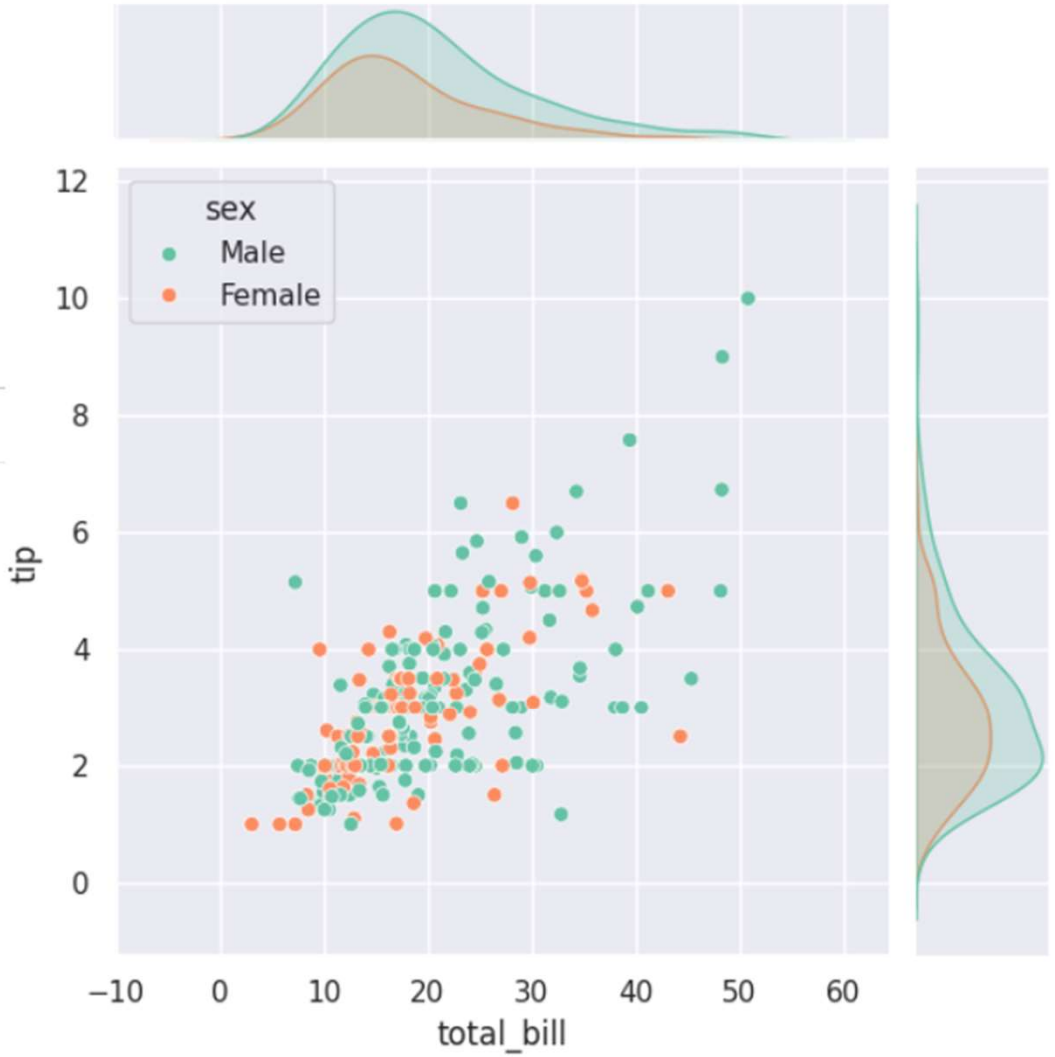
- Scatter plot with addition of distributions of values on both axes

```
sns.jointplot(data=tips, x='total_bill', y='tip')
```



- Scatter plot with populations distinguished by gender, with relative distributions of values

```
sns.jointplot(data=tips, x='total_bill', y='tip', hue='sex')
```




Q&A